

全国大学生数学建模竞赛论文题目

摘要

这里是全国大学生数学建模竞赛论文的展示，我接下来要通过**贪心算法**来解释，接下来你会看到很多内容。

针对问题一：首先简要介绍**工作原理**，以中心点为坐标坐标系。

关键字： 多边形覆盖 **线性拟合** 流程图

目录

一、问题的重述

1.1 问题的背景

无线电是科技信息时代的重要工具，广泛应用于新闻通讯、商业交互、军事联络等诸多领域。无线电干扰会影响通信设备及无线电系统的正常运行，快速定位并清除干扰源是频谱管理中的重要任务。传统方法分为初步监测、区域性排查、精准定位三阶段，其中可依据固定监测站、移动监测车或无人机进行大范围初筛，但精确定位主要依靠人工携带测向设备在目标区域内逐步排查，效率较低，且在危险环境下存在安全隐患。

某公司拟研发一个搭载于机器狗的干扰源自动定位清除系统，通过利用测向设备获取信号场强及方向等信息，自主搜索并定位干扰源。已知干扰源位于以原点为圆心、半径 1800 m 的圆形区域内，数量和具体位置均未知。以正东方向为 x 正半轴，正北方向为 y 轴正半轴建立平面直角坐标系。

1.2 问题的提出

问题 1： 目前存在坐标已知的若干检测点，且已知某干扰源关于这些检测点的示向度，以及用来确定位置的交会定位法。需建立算法计算出多边形定位区域的直径，并判断以此为直径的圆是否可以覆盖此定位区域。

问题 2： 已知某一个全方向发出信号的干扰源及其在一检测点的示向度，应建立数学模型确定第二个检测点的最优坐标范围。

问题 3： 在未知具体数目（10-16 个）的干扰源全方向发出干扰信号的情境下，配备干扰源自动定位清除系统的机器狗初始位置在原点，测向机初始频道为 1，求任务用时最短的策略及算法，并以“被清除干扰源个数的比例”和“平均定位清除时间”为评价标准，在模拟器的演练测试计算检验策略有效性。

问题 4： 在问题 3 的基础上，未知具体数目（10-16 个）的干扰源包含全向或定向（定向方向两侧各 90° ）两种类别，现在仍然求出检测与清除的策略与算法并检验有效性。

二、模型的假设

1. 假设不考虑信号接收源的示位偏差。
2. 假设干扰源的信号强度在各个方向上均匀分布。

3. 假设忽略测向机与干扰源的高度差异。

三、符号说明

符号	含义	单位
S_i	第 i 个检测点坐标	m
θ_i	第 i 个检测点的示向度	°
τ	测向误差界	°
$\mathcal{R}$	交会定位区域	—
D	定位区域的直径	m
R_{MEC}	最小覆盖圆半径	m

四、问题分析

4.1 问题 1 分析

首先，根据测向机存在一定测量误差这一条件，规定每个检测点对应的可能区域，即以检测点为顶点、测量方向为中心的一定角度范围内的区域。将所有检测点对应的区域取交集，就得到干扰源可能存在的共同区域。其次，为了研究这个区域的几何性质，我们将上述角度限制转化为直线和半平面约束，从而证明多个检测点形成的相交区域是一个闭的凸区域；在区域有界的情况下，它进一步表现为一个闭凸多边形。这样就可以把原本较复杂的定位问题转化为比较直观的平面几何问题。接着，我们进一步讨论这个定位区域是否一定是有限的。通过分析各个检测点测得方向之间的关系，判断它们是否存在共同的延伸方向，从而确定定位区域是否有界。最后，在定位区域有界的情况下，考虑区域内两点之间的最大距离。直径 = $\max_{i < j} \|v_i - v_j\| = \max_{i < j} \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ 顶点数小于或等于 $2n$ ，最终可以写为非线性规划的形式。运用 MATLAB 有两种方法：一种是旋转卡壳法，可以将 $O(m^2)$ 转化为 $O(n)$ ，还有一种是向量化暴力求解，适用于大数量多边形、少量顶点的情况，比普通的遍历算法更优。

首先规定相交区域，其次证明它是闭凸多边形，接着关于其有界性证明，最后证明距离的最大值在对角处取到。

4.2 问题 2 分析

问题二主要研究在首次测向结果已知的情况下，如何选择第二个检测点，使后续定位尽可能准确，同时兼顾检测点的移动距离。首先，根据第一检测点的测向误差和有效接收距离，确定干扰源可能存在的区域，并进一步反推出第二检测点的候选范围，保证第二站能够接收到信号。其次，以第二检测点对定位精度的改善程度作为主要评价指标，引入最小包围圆半径，衡量不同位置下最坏情况下的定位误差，同时将移动距离作为次要优化目标。然后，为降低计算量，采用“两阶段搜索”策略：第一阶段利用测向误差的线性化模型进行快速粗筛，并结合理论下界排除明显较差的候选点；第二阶段对保留下来的候选点建立真实几何定位区域，通过多边形求交和最小包围圆进行精确评价。最后，在保证定位精度基本最优的条件下，选择移动距离较短的检测点作为最终方案。

4.3 问题 3 分析

4.4 问题 4 分析

五、模型的建立与求解

5.1 问题 1：交会区域直径的计算

为了求得定位区域直径，我们首先需要通过交会定位法确定并表示出该区域。确定该区域后我们需要给出直径的算法，而对区域内每两点求距离计算量过于庞大，因此我们将在下文中通过确认定位区域的闭凸性和有界性证明它是闭凸多边形，随后证明直径必然是其对角线中的一条来大大减少计算量。最后我们将多种 Matlab 求解方法对比，选择了更加适合该题目的向量暴力求解方法。

5.1.1 交会定位区域的建立

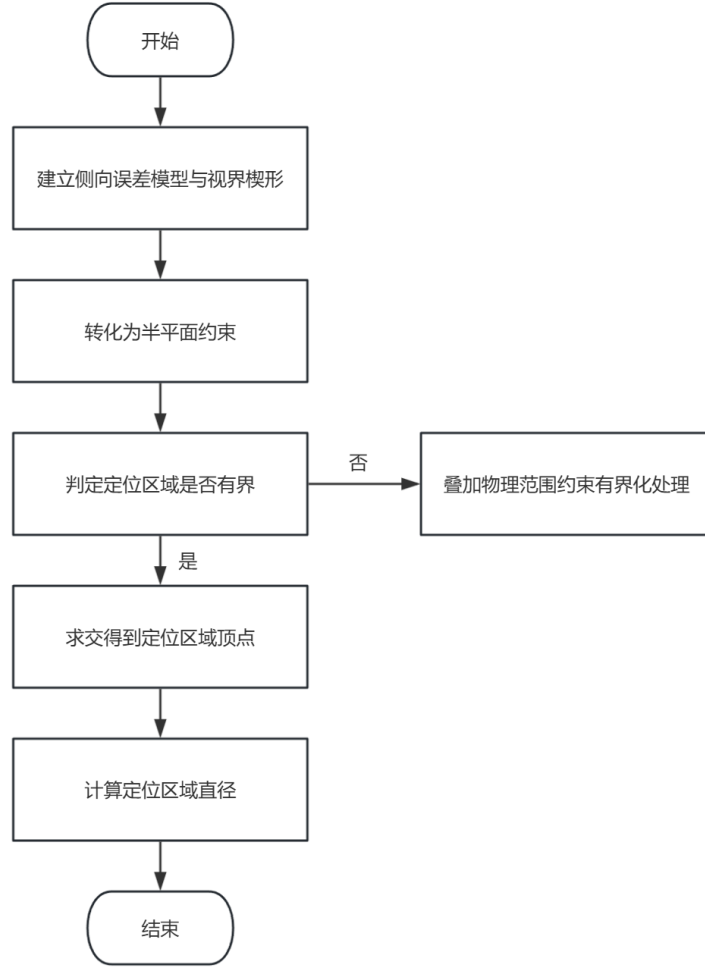


图 1 流程图

设共有 n 个检测点，其坐标分别为 $S_i = (x_i, y_i)$, $i = 1, 2, \dots, n$ 。对应测得的示向度为 $\theta_i \in [0^\circ, 360^\circ)$ 。根据附录 2，测向机的示向度与干扰源真实方位角之间的误差绝对值不超过 $\tau = 1^\circ$ 。设干扰源真实位置为 G ，则在第 i 个检测点处，其真实方位角 $\beta_i = \arg(G - S_i)$ 满足 $|\text{wrap}(\beta_i - \theta_i)| \leq \tau$ 。其中偏差的取值通过公式

$$\text{wrap}(x) = x - 360^\circ \left\lfloor \frac{x + 180^\circ}{360^\circ} \right\rfloor.$$

统一折算到 $(-180^\circ, 180^\circ]$ 区间内，利用 wrap 函数避免角度周期性带来的误判。

而对于每个检测点 S_i ，干扰源一定落在以 S_i 为顶点、以 $\theta_i - \tau$ 和 $\theta_i + \tau$ 为两条边界射线的角形区域内。我们把这块区域记为第 i 个检测点的视界楔形

$$W_i = \{P : P = S_i \text{ 或 } |\text{wrap}(\arg(P - S_i) - \theta_i)| \leq \tau\}. \quad (1)$$

交会定位区域实际上就是所有视界楔形的交集

$$\mathcal{R} = \bigcap_{i=1}^n W_i. \quad (2)$$

回到题设，干扰源的真实位置 G 必须同时落进每一个检测点的视界楔形中，即 $G \in W_i$ ($i = 1, \dots, n$)，于是立即可以得到：

$$G \in \mathcal{R}. \quad (3)$$

也就是说， $\mathcal{R}$ 正是干扰源位置的理论可行域。

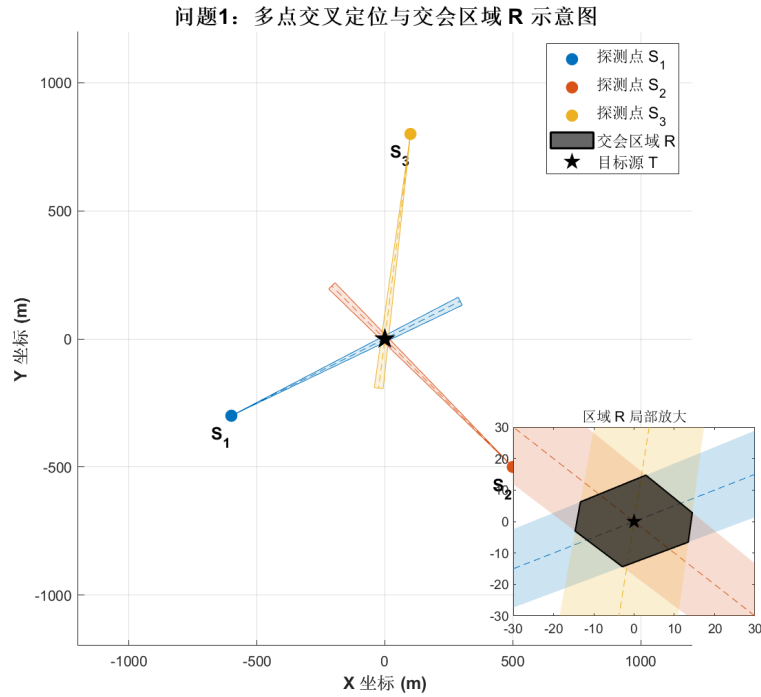


图 2 多点定位交叉和交会区域示意图

5.1.2 定位区域的闭凸性证明

为了便于进一步计算定位区域的几何性质，对每一个检测点 S_i 建立局部正交坐标系。定义

$$u_i = (\cos \theta_i, \sin \theta_i), \quad v_i = (-\sin \theta_i, \cos \theta_i), \quad (4)$$

其中 u_i 为测得方向的单位向量， v_i 为其逆时针旋转 90° 后的单位向量。

对任意待测位置 P ，记 $d = P - S_i$ ，再把它在这组局部基下拆开，即

$$d = \xi_i u_i + \eta_i v_i, \quad (5)$$

其中 $\xi_i = d \cdot u_i$, $\eta_i = d \cdot v_i$ 。 ξ_i 表示点 P 沿示向度方向的纵向距离, η_i 表示偏离示向度方向的横向距离。

当 $d \neq 0$ 且 $\xi_i > 0$ 时, 方位角和 (ξ_i, η_i) 之间有以下关系:

$$\arg(d) - \theta_i = \arctan \frac{\eta_i}{\xi_i}. \quad (6)$$

两边取绝对值, 测向误差约束就化成了对横向与纵向距离之比的限制

$$|\arg(d) - \theta_i| \leq \tau \iff \left| \frac{\eta_i}{\xi_i} \right| \leq \tan \tau. \quad (7)$$

再把目标一定位于检测点前方这一条件 $\xi_i \geq 0$ 一并纳入, 整理后可得

$$W_i = \{S_i + \xi u_i + \eta v_i : \xi \geq 0, |\eta| \leq \xi \tan \tau\}. \quad (8)$$

进一步地, $|\eta_i| \leq \xi_i \tan \tau$ 等价于

$$\eta_i \leq \xi_i \tan \tau, \quad -\eta_i \leq \xi_i \tan \tau.$$

由于 ξ_i 和 η_i 均为位置坐标 $P = (x, y)$ 的线性函数, 因此上面这两个条件都可以直接写成半平面不等式的形式。

具体地, 令

$$e_i^\pm = (\cos(\theta_i \pm \tau), \sin(\theta_i \pm \tau)), \quad (9)$$

则楔形两条边界射线分别为

$$S_i + t e_i^\pm, \quad t \geq 0.$$

这两条边界所在直线的交点, 即楔顶, 为

$$A_i = S_i + \frac{1}{\cos \tau} u_i. \quad (10)$$

定义单位法向量

$$n_i^\pm = (\sin(\theta_i \mp \tau), -\cos(\theta_i \mp \tau)),$$

则视界楔形可表示为两个半平面的交:

$$W_i = \{P : n_i^+ \cdot P \leq n_i^+ \cdot A_i\} \cap \{P : n_i^- \cdot P \leq n_i^- \cdot A_i\}. \quad (11)$$

因此, 交会定位区域可以进一步表示为

$$\mathcal{R} = \bigcap_{j=1}^{2n} \{P : n_j \cdot P \leq c_j\}. \quad (12)$$

而由于有限个闭半平面的交仍为闭凸集，因此 $\mathcal{R}$ 一定是闭凸区域。

5.1.3 定位区域有界性的判定

$\mathcal{R}$ 是闭凸区域：当其有界时， $\mathcal{R}$ 即为一个凸多边形，且边界至多包含 $2n$ 条边。而凸多边形的直径是易求的，因此我们将讨论 $\mathcal{R}$ 是否有界。

由于测向误差的存在，当各检测点的示向度过于接近时，多个视界楔形可能存在共同的延伸方向，使得交会定位区域向无穷远处扩张：

设非零向量 ξ 是 $\mathcal{R}$ 的一个逃逸方向，即存在 $P_0 \in \mathcal{R}$ ，使得 $P_0 + t\xi \in \mathcal{R}$ ($\forall t \geq 0$)。对于第 i 个视界楔形来说，它能无限延伸的方向，恰好是那些与示向度 θ_i 夹角不超过 τ 的方向，即

$$|\text{wrap}(\theta_i - \arg \xi)| \leq \tau \quad (13)$$

换言之， $\mathcal{R}$ 存在非零逃逸方向，等价于存在某个方向 φ 能够覆盖所有检测点的示向度：

$$|\text{wrap}(\theta_i - \varphi)| \leq \tau, \quad i = 1, \dots, n.$$

把各个示向度看成单位圆周上的点，上述条件正好对应“它们能被一条长度为 2τ 的弧覆盖”。为定量描述，定义这些方向点的最小包围弧长度为：

$$\alpha(\Theta) = 360^\circ - \max_k (\theta_{(k+1)} - \theta_{(k)})_{360}, \quad (14)$$

其中 $\theta_{(k)}$ 为示向度升序排列后的结果，下标按圆周循环， $(\cdot)_{360}$ 表示模 360° 后落入 $[0^\circ, 360^\circ)$ 的环间隔。

因此可知，若所有示向度都能塞进某条长度为 2τ 的弧里，就存在共同逃逸方向，定位区域无界；反之，只要最小包围弧长度超过 2τ ，就找不到共同逃逸方向，区域必然有界。于是得到有界性判据

$$\mathcal{R} \text{ 有界} \iff \alpha(\Theta) > 2\tau. \quad (15)$$

退化到只有两个检测点的情形，先算出两示向度的最小夹角

$$\Delta\theta = |\text{wrap}(\theta_2 - \theta_1)| < 180^\circ. \quad (16)$$

此时显然有 $\alpha(\Theta) = \Delta\theta$ ，代入上面的判据，立刻得到

$$\mathcal{R} \text{ 有界} \iff \Delta\theta > 2\tau = 2^\circ. \quad (17)$$

也就是说，当两个检测点观测到的方向夹角不超过 2° 时，交会区域可能沿共同方向无限延伸，不能直接定义有限直径。遇到这种情况，则需给定位区域补上有界化条件。

5.1.4 定位区域直径的计算

当上述有界性条件成立时它就是紧凸多边形，直径自然有定义，即

$$D = \text{diam } \mathcal{R} = \max_{P, Q \in \mathcal{R}} |P - Q|. \quad (18)$$

然而直接对区域上所有点对求最大距离仍然十分困难。但我们可以根据 $\mathcal{R}$ 是凸多边形，将其连续优化进一步压成有限个顶点对的比较。

设 $\mathcal{R}$ 的顶点按逆时针方向排列为 $V_1, V_2, \dots, V_m$ 。固定任意 $Q \in \mathcal{R}$ ，函数 $f(P) = |P - Q|$ 关于 P 为凸函数。若 P 落在某条边的内部，可将其写成该边两个端点的凸组合，于是由三角不等式，至少有一个端点到 Q 的距离不小于 P 到 Q 的距离。这说明最大值一定能在顶点处取到，即：

$$D = \max_{1 \leq i < j \leq m} |V_i - V_j|. \quad (19)$$

于是，问题一中定位区域直径的计算可以归纳为以下步骤：

1. 根据每个检测点的坐标 S_i 、示向度 θ_i 和误差界 τ ，建立两个半平面约束，共得到至多 $2n$ 个半平面；
2. 对这些半平面进行求交，获得定位区域 $\mathcal{R}$ 的顶点 $V_1, \dots, V_m$ ($m \leq 2n$)；
3. 枚举任意两个顶点，计算 $d_{ij} = |V_i - V_j|$ ，取最大值作为定位区域直径

$$D = \max_{i < j} d_{ij}.$$

5.1.5 Matlab 求解方法

在对半平面求交得到 $\mathcal{R}$ 的顶点 $V_1, \dots, V_m$ ($m \leq 2n$) 后，我们需要求解以下非线性规划问题：

$$\begin{aligned} & \max_{V_i, V_j} \|V_i - V_j\|_2 \\ & \text{s.t.} \quad \begin{cases} V_i, V_j \in \mathcal{R}, \\ \mathcal{R} = \bigcap_{k=1}^{2n} \{X : n_k \cdot X \leq c_k\}, \\ n_k \cdot X \leq c_k \text{ 为半平面的函数表达式.} \end{cases} \end{aligned} \quad (20)$$

若直接枚举顶点对，直径计算的复杂度为 $O(m^2)$ ，再代入 $m \leq 2n$ ，可进一步写为 $O(n^2)$ 。当顶点已按凸多边形边界顺序排列时，也可以进一步利用旋转卡壳算法将直径计算降至 $O(m)$ ，而考虑到问题中检测点数量一般较少，因此我们真正需要的是可以快速求解大数量但边数较少的闭凸多边形直径的算法；旋转卡壳算法适用于检测点数量较少，但边数较多的闭凸多边形直径的计算，虽然降低了复杂度但速度并没有提升太多且

其代码容易出错，经综合考量后我们决定采用代码更为稳定，适用于求解大数量数据的向量化求解算法。

但考虑到问题中检测点数量一般较少，因此我们真正需要的是可以快速求解大数量但边数较少的闭凸多边形直径的算法；而旋转卡壳算法适用于检测点数量较少，但边数较多的闭凸多边形直径的计算，虽然降低了复杂度但速度并没有提升太多且其代码容易出错，经综合考量后我们决定采用代码更为稳定，适用于求解大数量数据的向量化求解算法。

5.1.6 以定位区域直径为直径的圆能否覆盖定位区域

问题一的第二部分要回答的是：以定位区域直径 D 为直径的圆，是否一定能把整个定位区域罩住？

为此先引入集合 K 的最小覆盖圆半径

$$R_{\text{MEC}}(K) = \min_C \max_{P \in K} |C - P|. \quad (21)$$

它衡量的是“用圆罩住 K 所需的最小代价”。注意到区域直径为 D ，就意味着存在一对点 $A, B \in K$ 满足 $|A - B| = D$ 。现在任取一个能覆盖 K 的圆，设圆心为 C 、半径为 R ，由三角不等式直接得到

$$D = |A - B| \leq |A - C| + |C - B| \leq 2R. \quad (22)$$

也就是说，任何覆盖圆的半径都逃不过 $D/2$ 这个下界，故

$$R_{\text{MEC}}(K) \geq \frac{D}{2}. \quad (23)$$

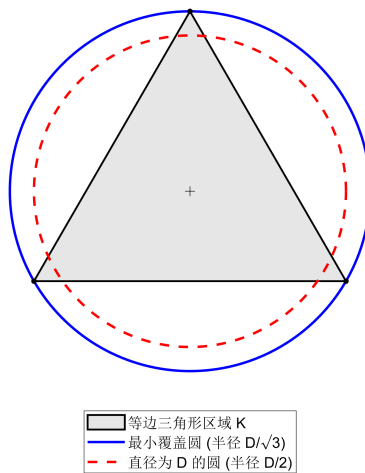


图3 等边三角形反例图

但这里要特别注意，上式一般不能取等号，也就不能保证存在一个直径恰好为 D 的圆罩住整个区域。为了说明这一点，可以取边长为 D 的等边三角形作为反例：它的直径确实是 $\text{diam } K = D$ ，可它的最小覆盖圆是外接圆，半径

$$R_{\text{MEC}}(K) = \frac{D}{\sqrt{3}}.$$

于是 $2R_{\text{MEC}}(K) = \frac{2}{\sqrt{3}}D > D$ 。这恰好说明：以定位区域直径 D 为直径的圆一般不能覆盖整个定位区域。

再进一步，由平面上的 Jung 定理，任意直径为 D 的有界集合都满足：

$$\frac{D}{2} \leq R_{\text{MEC}}(K) \leq \frac{D}{\sqrt{3}}, \quad (24)$$

其中等边三角形达到上界。因此，如果需要构造能够保证覆盖整个定位区域的圆形搜索域，其严格的尺度应采用最小覆盖圆直径 $2R_{\text{MEC}}$ ，而不能简单地将定位区域直径 D 直接作为圆的直径。

5.2 问题二：第二检测点的选择

5.2.1 确定候选区域

为求得检测点 2 所在区域，我们将先选出候选区域，保证候选区域内所有点可以扫描到干扰源所有可能位置，然后再于候选区域内找到检测点 2 的最优位置。

以首个检测点为原点，以首次测得的示向度为 x 轴正向，建立局部坐标系。设第二检测点为 $P = (x, y)$ ，干扰源为 $G = (u, v)$ ，误差半角为 $\tau = 1^\circ = \pi/180$ 。干扰源在两次观测之间保持不变。

首次正常观测后，干扰源可能的位置为：

$$E_0 = \left\{ (u, v) : u \geq 0, |v| \leq u \tan \tau, 5 < \sqrt{u^2 + v^2} \leq 1500 \right\}. \quad (25)$$

为保证第二站接收到信号，利用有效接收半径至少为 1000 米，选取保守候选区域

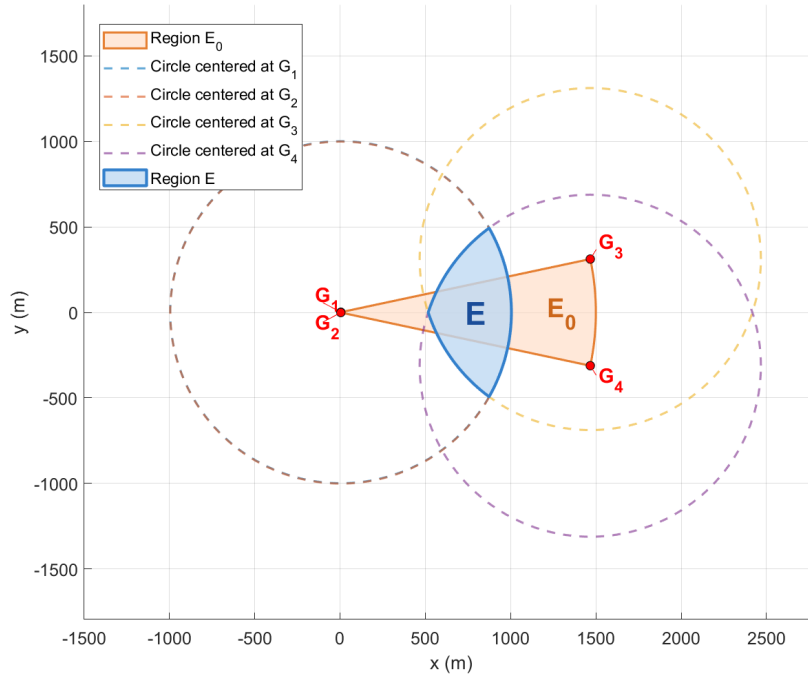
$$E = \left\{ P : \sup_{G \in E_0} \|P - G\| \leq 1000 \right\} \quad (26)$$

为满足 $\sup_{G \in E_0} \|P - G\| \leq 1000$ ，等价于要求距离函数 $f(G) = \|P - G\|$ 在区域 E_0 上的最大值不超过 1000。而距离函数 $f(G)$ 是关于 G 的连续凸函数，依据凸函数的极值性质，其在有界闭区域上的最大值必然在该区域的顶点处取得。

具体到扇形区域 E_0 ：在极坐标下固定半径 r 时， $f(G)$ 沿圆弧没有内部极大值，最大值只能在极角边界 $\pm\tau$ 处取得；固定极角时， $f(G)$ 关于 r 也是凸的，最大值只能在极径两端取得。因此， $f(G)$ 在 E_0 上的最大值必然在四个角点（即 $r \in \{5, 1500\}$ 且极角在 $\pm\tau$ 边界处的 G_1, G_2, G_3, G_4 四个点）处取得。

所以，只需保证这四个顶点到 P 的距离均小于等于 1000 米，即可确保 E_0 内所有点均满足接收要求。这就将无穷多个点 G 的约束严格简化为了四个点的距离约束。

$$\begin{aligned}
 E &= \bigcap_{i=1}^4 \{P : \|P - G_i\| \leq 1000\} \\
 &= \bigcap_{\substack{r \in \{5, 1500\} \\ s \in \{-1, 1\}}} \{(x, y) : (x - r \cos \tau)^2 + (y - sr \sin \tau)^2 \leq 1000^2\}.
 \end{aligned} \tag{27}$$



5.2.2 以最差包围半径衡量精度

仅仅有待测区域的确定还不够，我们还需要知道该区域内每个点所交会产生图形的最差包围圆半径的大小来衡量这个位置定位的精度。

记 $W(P; \beta, \tau)$ 为检测点 P 、中心方向 β 、半角 τ 的前向楔形。正常读数 β 对应的定位区域为：

$$J(P, \beta) = E_0 \cap W(P; \beta, \tau) \cap \{G : \|G - P\| > 5\}. \tag{28}$$

距离源不超过 5 米时，仪器无法给出示向度，应直接采用光学定位与清除。楔形约束还可改写为两条线性不等式：

$$\begin{cases} \sin(\beta - \tau)(u - x) - \cos(\beta - \tau)(v - y) \leq 0, \\ -\sin(\beta + \tau)(u - x) + \cos(\beta + \tau)(v - y) \leq 0. \end{cases} \tag{29}$$

由于还包含圆弧和近距离排除区, J 不一定是四边形。

对非空集合 A , 定义最小包围圆半径 $\rho(A) = \min_c \sup_{G \in A} \|G - c\|$, 并用闭包计算相同半径。所有可能正常读数构成集合 $\Theta(P)$, 即:

$$\Theta(P) = \bigcup_{\substack{G \in E_0 \\ \|G-P\| > 5}} [\arg(G-P) - \tau, \arg(G-P) + \tau], \quad (30)$$

在此基础上, 我们定义检测点 P 的“最差包围半径” $f_1(P)$ 为:

$$f_1(P) = \sup_{\beta \in \Theta(P)} \rho(J(P, \beta)). \quad (31)$$

这里 $f_1(P)$ 评估了所有可能测得的示向度 β 所对应定位区域半径的最大值, 因此该指标同时考虑了干扰源位置未知性以及测角误差, 代表了最差情况的结果。

最理想情况下, 若 $\rho(J(P, \beta)) \leq 20$, 走到相应圆心即可保证光学定位成功。

然而对完整扇区, 实际上不存在满足 $f_1(P) \leq 20$ 的 $P \in E$ 。令

$$C = \left(\frac{1500}{1 + \sin \tau}, 0 \right), \quad r_* = \frac{1500 \sin \tau}{1 + \sin \tau} \approx 25.730.$$

圆盘 $B(C, r_*)$ 内切于扇区, 且完全位于首站 5 米排除区之外。由 $P \in E$ 得 $\|P\| \leq 1005$, 而 $\|C\| - r_* > 1448$, 所以从 P 指向 C 的射线穿过该圆盘的一条完整直径, 直径上的点距 P 均大于 5 米且与同一读数相容。因此:

$$f_1(P) \geq r_* > 20, \quad P \in E.$$

这给出不可行性的下界证明, 并不表示最优半径能达到 r_* 。因此对每个点位置打分来衡量优劣是必要的。

由于原始定义涉及的不可解圆弧与复杂多边形求交, 计算量巨大, 我们首先引入线性化近似, 对候选区域进行粗略打分以筛选大部分无效点, 然后再对剩余候选点进行高精度精确打分。

5.2.3 第一阶段打分（粗筛）

直接对整个候选区域 E 做高精度几何求交开销极大。因此, 我们首先构造一个低成本的极速打分指标, 对所有候选点进行近似排序。

由于测向方程中包含非线性的三角函数, 直接求解精确的边界交点十分复杂。考虑到实际设备允许的最大测角误差 $\tau = 1^\circ$ 非常小, 因此可以采用小角度近似 ($\sin \tau \approx \tau$ 且 $\cos \tau \approx 1$) 来简化问题, 我们在首条中心射线取参考源 $G = (r, 0)$, 对测向方程在 G 点附近作一阶泰勒展开线性化, 使原本复杂的非线性定位区域边界被简化为了四条直线, 得到一个规范的“位置误差平行四边形”:

$$|\Delta v| \leq r\tau, \quad |y\Delta u + (r-x)\Delta v| \leq ((r-x)^2 + y^2)\tau.$$

其包围半径（即粗筛阶段的评分公式）为：

$$\hat{\rho}(r; P) = \tau \sqrt{r^2 + \frac{((r-x)^2 + y^2 + r|r-x|)^2}{y^2}}, \quad y \neq 0. \quad (32)$$

我们发现该近似模型中，径向最坏误差必然在区间的两端（ $r = 5$ 或 $r = 1500$ ）取得。具体的推导过程如下：令

$$H(r) = (r-x)^2 + y^2 + r|r-x| = \max\{x^2 + y^2 - xr, 2r^2 - 3xr + x^2 + y^2\}, \quad r \geq 0.$$

首先，函数 $f_1(r) = x^2 + y^2 - xr$ 是关于 r 的线性函数，因而是凸函数；

函数 $f_2(r) = 2r^2 - 3xr + x^2 + y^2$ 是关于 r 的二次函数，其二阶导数为 $4 > 0$ ，因而是严格凸函数。

由于两个凸函数的逐点最大值仍然是凸函数，所以 $H(r) = \max\{f_1(r), f_2(r)\}$ 必定是非负的凸函数。

其次，对于给定的候选点 $P(x, y)$ ，打分公式可写为二维向量的欧几里得范数：

$$\hat{\rho}(r; P) = \tau \left\| \left(r, \frac{H(r)}{|y|} \right) \right\|_2. \quad (33)$$

由欧氏范数在非负象限上的单调性，以及 $r \mapsto r$ （线性）和 $r \mapsto H(r)/|y|$ （凸且非负）的凸性组合可知，复合函数 $\hat{\rho}(r; P)$ 严格保持了关于 r 的凸性。

根据凸优化理论的基本定理：凸函数在闭区间 $[5, 1500]$ 上的最大值必然在区间的边界端点处取得。因此，我们严密推导出第一阶段候选点 P 的最终打分为：

$$\hat{f}_1(P) = \max\{\hat{\rho}(5; P), \hat{\rho}(1500; P)\}. \quad (34)$$

这一端点定理极大地减少了搜索空间。但该结论仅适用于线性化指标，不适用于真实的几何区域模型。因此 $\hat{f}_1$ 仅用于第一阶段极速打分排序用以降低计算量； $y = 0$ 的奇异点则直接交由第二阶段模型评价。

另从 E_0 中取有限个见证点 G_i 。若两点距 P 均大于 5 米，且从 P 看去的夹角不超过 2τ ，则它们可产生同一正常读数，因而

$$\ell_{\text{pair}}(P) = \max_{\substack{i < j, \|G_i - P\|, \|G_j - P\| > 5 \\ \angle(G_i - P, G_j - P) \leq 2\tau}} \frac{\|G_i - G_j\|}{2} \leq f_1(P). \quad (35)$$

无符合条件的点对时取下界 0。设已有某候选点的评分上界为 U_{best} ，若 $\ell_{\text{pair}}(P) > U_{\text{best}} + \tau$ ，则可排除该点，不予进一步细算。

5.2.4 第二阶段打分（精评）

在经由第一阶段的初步筛选后，对于剩余的候选点 P ，用源域的凸外多边形确定所有可能方向的覆盖区间，再向两侧扩展 τ 。若 P 位于该多边形内，取整个圆周。这样既能处理跨越 0° 的读数，也能包含所有可能正常观测。

将覆盖区间分成若干个连续的细分角度小区间 $I_j = [\beta_j - \eta_j, \beta_j + \eta_j]$ ，其中 β_j 是第 j 个小区间的中心角度， η_j 是该区间的半宽。

若满足 $\tau + \eta_j < \pi/2$ ，则对于任意属于该小区间内的读数 $\beta \in I_j$ ，其真实的定位区域均有如下包含关系：

$$J(P, \beta) \subseteq E_0 \cap W(P; \beta_j, \tau + \eta_j) \cap \{G : \|G - P\| > 5\}. \quad (36)$$

为了有效计算最差包围半径 $f_1(P)$ ，我们采用内外多边形近似来构建其上下界：

对于右侧的区域，构造一个外接多边形近似记为 K_j^+ ，它的最小包围圆半径 $\rho(K_j^+)$ 必然大于真实区域的半径，从而为该小区间 I_j 提供了包围半径的局部上界。

同理，在小区间的真实定位区域 $J(P, \beta_j)$ 内构造一个内接多边形近似记为 K_j^- ，它提供了局部下界。

综合所有划分的小区间，并结合前文已求得的理论下界，我们便可以得到评估候选点 P 的全局下界 $L(P)$ 与全局上界 $U(P)$ ：

$$L(P) = \max \left\{ \ell_{\text{pair}}(P), r_*, \max_{\text{已采样 } j} \rho(K_j^-) \right\} \leq f_1(P) \leq U(P) = \max_j \rho(K_j^+). \quad (37)$$

由于求精确的非线性交点非常耗时，通过这种上下界放缩的机制，我们只需不断细分角度区间，当 $U(P)$ 和 $L(P)$ 足够接近时，即可认为我们以极高的精度逼近了真实的最差包围半径 $f_1(P)$ 。

注意内外近似须保持包含方向：源域圆弧用连弦构造子集、相邻切线构造超集；扣除近源圆孔时则相反，内近似扣除外接孔，外近似只扣除内接孔。

对正 n 边形，半径为 a 的内接孔的边心距为 $a \cos(\pi/n)$ 。裁剪后，我们需要对各凸片的全部顶点求“最小包围圆”（MEC），得到相应半径与圆心，而这就是我们第二阶段精评打分的绝对依据。

最小包围圆（MEC）算法原理与公式：给定多边形的顶点集合 $S = \{V_1, V_2, \dots, V_m\}$ ，最小包围圆问题是寻找一个半径最小的圆，使得集合 S 中的所有点都在该圆内或圆周上。其数学模型为求解如下最优化问题：

$$\begin{aligned} \min_{c, \rho} \quad & \rho \\ \text{s.t.} \quad & \|V_i - c\| \leq \rho, \quad \forall i = 1, 2, \dots, m \end{aligned} \quad (38)$$

根据计算几何原理，该最优包围圆是唯一存在的，并且它完全由集合 S 中位于其边界上的 2 或 3 个点来唯一确定。

在程序实现中，我们将多边形的顶点离散化后，直接计算其极小极值包围圆的半径 ρ ，这也是我们在给定测向角 β 时所面临的最差误差半径。

我们采用二分上界最大的角度箱策略，直到 $U - L$ 达到指定容差或下界超过淘汰阈值。程序分别记录收敛、淘汰与预算耗尽状态，并缓存角度箱以供后续续算。这里的上下界来自集合包含关系，具体实现使用双精度浮点与数值余量。

在圆弧与孔的近似分辨率固定时，极小容差可能无法达到，此时应提高几何分辨率。

5.2.5 精度优先、兼顾路程的两阶段策略

为寻找最优第二检测点 P ，我们设计了“精度优先、兼顾路程”的两阶段搜索策略。首先在 E 内生成网格点与随机点，利用粗评指标 $\hat{f}_1$ 筛选出部分排名靠前且分布较广的点作为搜索起点；接着利用前文下界进行提前淘汰，从这些起点开展带 E 约束的八方向模式搜索。最后，优先对上界较小的候选点进行精评，并实时检验路程较短的点（精评总数设限以避免全域扫描）。

该策略的理论目标是先最小化最差包围半径 f_1 ，再在允许的微小精度让步 ε 内最小化路程。其数学规划模型为：

$$\begin{aligned} \min_P \quad & f_2(P) = \|P\| \\ \text{s.t.} \quad & \begin{cases} f_1(P) \leq m + \varepsilon, \\ m = \inf_{P' \in E} f_1(P'), \\ P \in E. \end{cases} \end{aligned} \quad (39)$$

实际搜索得到的是有限个精评点组成的集合 $\mathcal{C}$ 。令

$$\hat{m} = \min_{P \in \mathcal{C}} U(P), \quad \hat{\mathcal{F}}_\varepsilon = \{P \in \mathcal{C} : U(P) \leq \hat{m} + \varepsilon\}, \quad S \in \arg \min_{P \in \hat{\mathcal{F}}_\varepsilon} \|P\|.$$

$\mathcal{C}$ 只包含达到精评容差的点。由此得到相对于当前数值最好结果的优良候选点集，移动时间为 $\|S\|/5$ 。

5.2.6 数值结果与适用范围

我们采用误差半角 1° ，粗评间隙 2 米，精评间隙 0.1 米。首次扇区的 2° 外圆弧分为 64 段，近源孔采用 64 边形。默认生成 411 个候选点，最多粗评 32 个点，局部搜索后最多精评 24 个点，粗评和精评分别限制为每点 160 和 900 次包围圆计算。在允许的微小精度让步 $\varepsilon = 1$ 米时的结果见表 ??：

表 1 完整扇区模型下的候选点比较，长度单位为米

选择依据	坐标 (x, y)	评分区间 $[L, U]$	路程
近似指标最小的候选点	(818.47, 580.15)	[55.639, 55.739]	1003.23
精评上界最小的候选点	(818.47, 580.15)	[55.639, 55.739]	1003.23
兼顾路程的选择	(799.22, -589.98)	[56.501, 56.571]	993.39

本次搜索的最小精评上界为 55.739 米，对应评分区间 [55.639, 55.739] 米。兼顾路程后选取 $S = (799.221, -589.98)$ ，其评分区间为 [56.501, 56.571] 米，路程为 993.393 米，移动时间约 198.679 秒。

候选区域的整体分布、粗候选点与精评点的相对位置，以及最终入选点的几何关系，如图 ?? 所示。

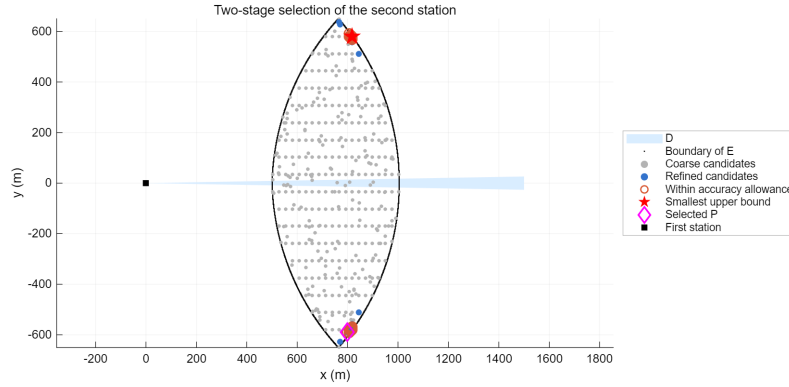


图 4 候选点概览图

此外，为了寻找严格满足精度容差的连续可行区域，我们在包含离散优良点集的局部候选框内生成了高密度网格，并利用前述评价函数进行了严密求交与测算。结果成功绘制出了满足 $U(P) \leq \hat{m} + \varepsilon$ 的连续区域边界及其评价热力图，如图 ?? 所示。

图中的热力颜色深浅反映了对应位置最差包围半径的大小，界定了最优靶区的局部边界。经严格测算，上下两侧完全满足精度容差的连续可行区域总面积约为 1730.05 平方米，这为无人机的最终移动与降落定位提供了足够的容错空间。

所选点在一个较差采样读数下，其定位区域与外近似包围圆的覆盖关系，如图 ?? 所示。

最优精评候选点的评分上下界随计算推进的收敛过程，如图 ?? 所示，展示最优精评候选点的评分上下界及区间宽度随计算推进的变化。

该方法的主要限制在于外层仍采用有限候选与局部搜索，因此 $\hat{m}$ 不是经过证明的全局最优值。若需要第三次观测，可将实际第二次读数得到的 J 作为新的源域继续选点；后续任务还应计入前往光学检测位置及清除所需的时间。

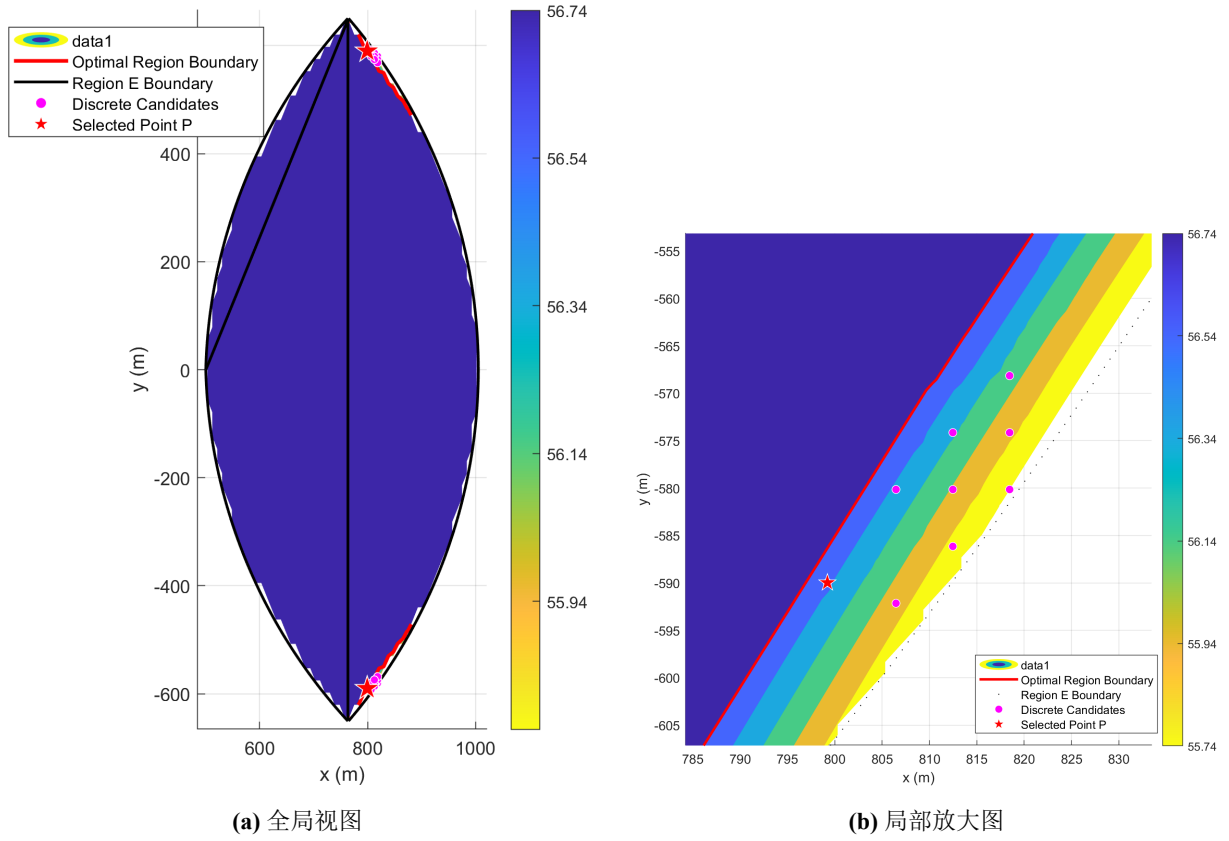
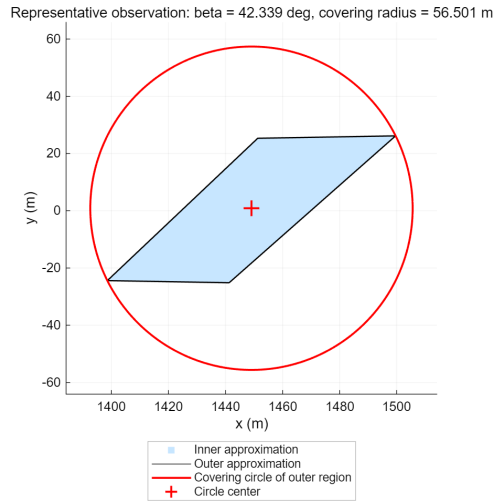


图 5 严格满足精度容差的连续可行区域热力图



5.3 问题 3：第三检测点的选择

关于干扰源在被机器狗检测与清除过程中大概有四种状态，如下表。对于编号为 04 的状态，即干扰源无法精确定位的状态，可以使用问题 2 的模型解决。如果存在某些干扰源在 $M_i \cap M_j$ 中，则可以在 S_i, S_j 分别探测形成交叉，其中有 $v_t \in M_i \cap M_j$ 。到某些干扰源予以清除时可以再次探测形成交叉，即 $d \leq 20m$ 时。若某干扰源 v_t 不在其他

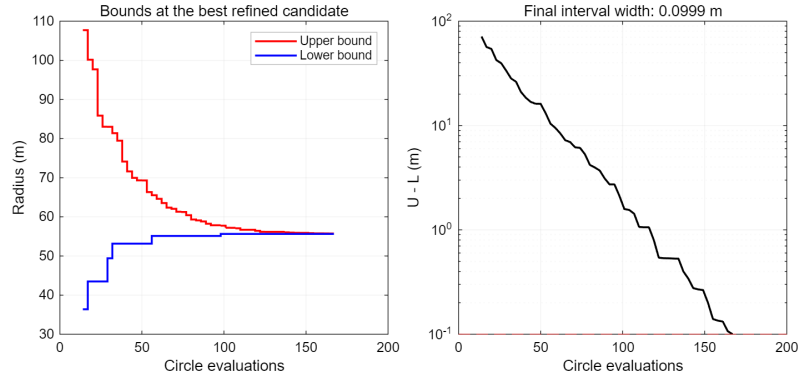


图 7 上下界收敛图

表 2 干扰源状态分类表

种类	干扰源状态
01	未被检测到
02	直径已小于 20 m, 可以清除
03	已被清除
04	无法精确定位

位置探测区域, 则将其频道不予考虑, 即 $\forall i, j, v_t \notin M_i \cap M_j$

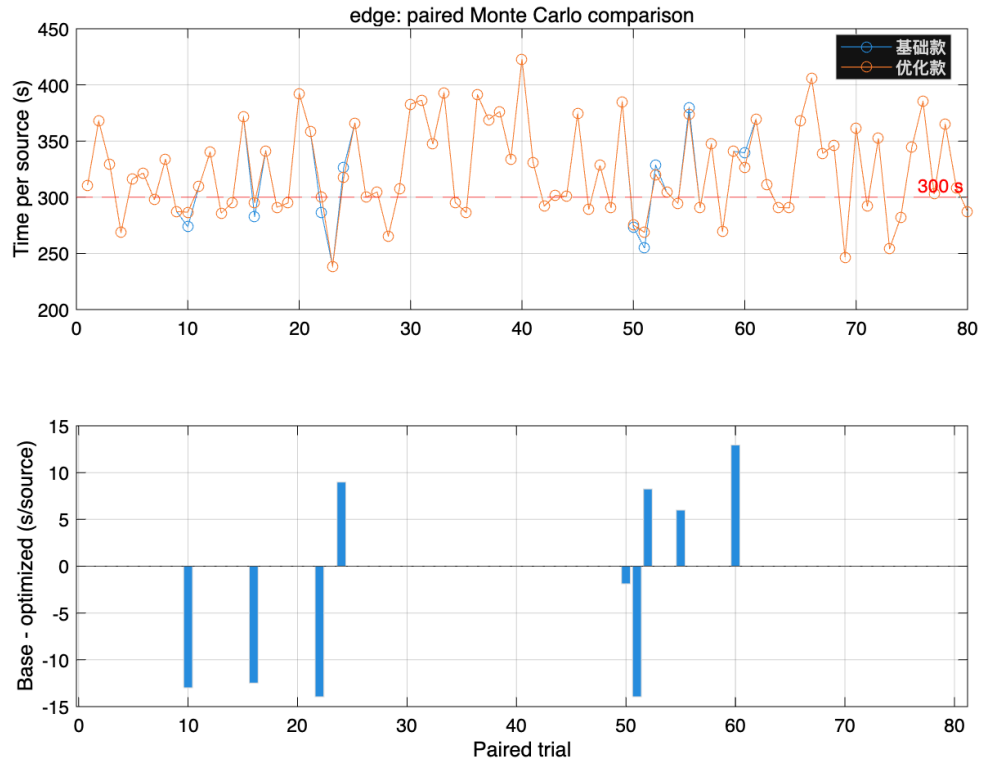


图 8 问题 3 边界情形下的比较结果

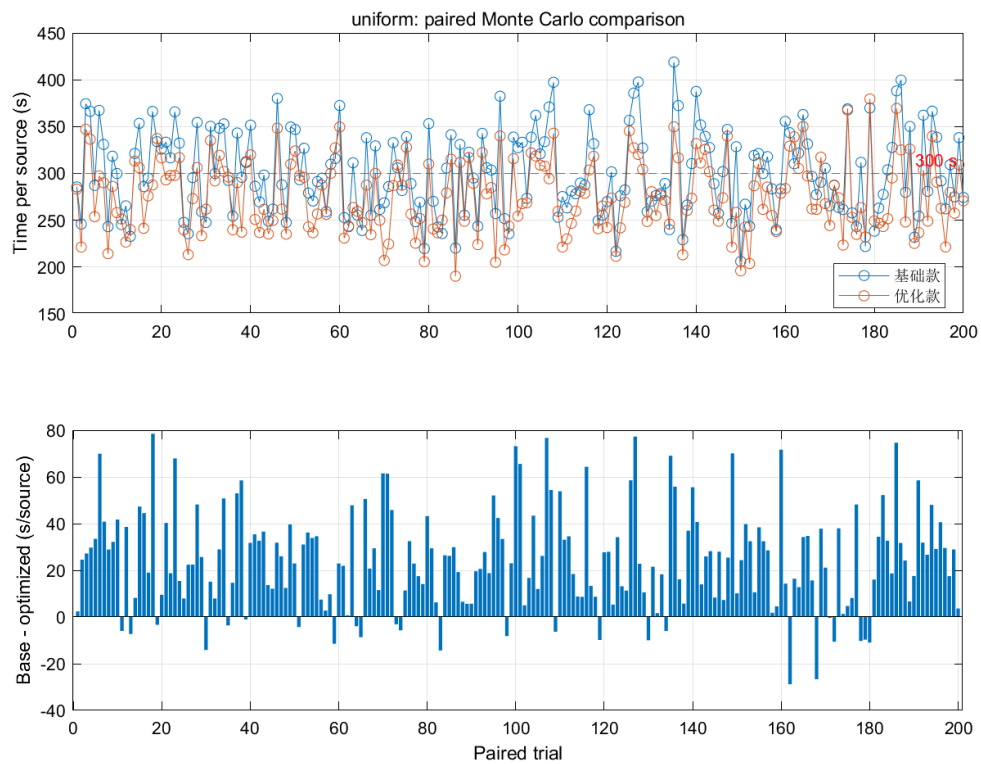


图 9 均匀情形下的比较结果

5.4 问题 4

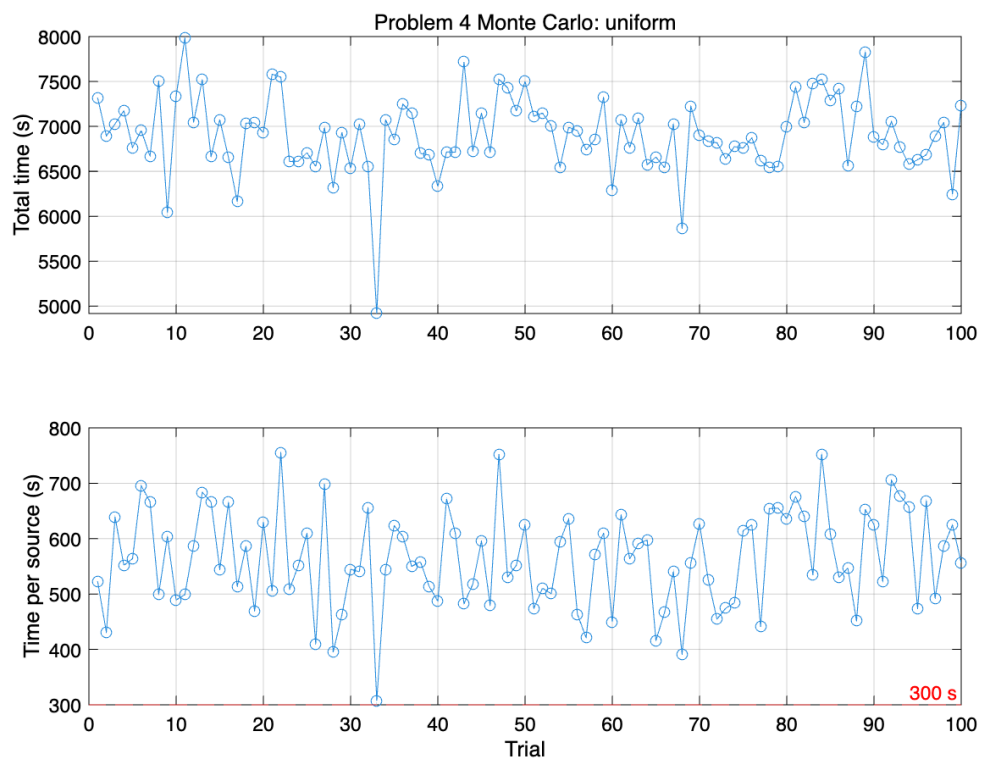


图 10 均匀情形下的问题 4 结果比较

六、模型的评价、改进与推广

七、AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于【简要用途，如语言润色、代码调试等】，详细使用情况见支撑材料。

参考文献

- [1] 刘海洋. $\text{\LaTeX}$ 入门[J]. 电子工业出版社, 北京, 2013.
- [2] 全国大学生数学建模竞赛论文格式规范 (2020 年 8 月 25 日修改).
- [1] <https://www.latexstudio.net>

附录 A 模板所用的宏包

表 3 宏包罗列

模板中已经加载的宏包				
ambsy	amsfonts	amsgen	amsmath	amsopn
amssymb	amstext	appendix	array	atbegshi
atveryend	auxhook	bigdelim	bigintcalc	bigstrut
bitset	bm	booktabs	calc	caption
caption3	CJKfntef	cprotect	ctex	ctexhook
ctexpatch	enumitem	etexcmds	etoolbox	everysel
expl3	fix-cm	fontenc	fontspec	fontspec-xetex
geometry	getttitlestring	graphics	graphicx	hobsub
hobsub-generic	hobsub-hyperref	hopatch	hxdetex	hycolor
hyperref	ifluatex	ifpdf	ifthen	ifvtx
ifxetex	indentfirst	inwarerr	intcalc	keyval
kvdefinekeys	kvoptions	kvsetkeys	l3keys2e	letltxmacro
listings	longtable	lstmisc	ltxcaption	ltxcmds
multirow	nameref	pdfescape	pdftexcmds	refcount
rerunfilecheck	stringenc	suffix	titletoc	tocloft
trig	ulem	uniquecounter	url	xcolor
xcolor-patch	xeCJK	xeCJKfntef	xeCJK-listings	xparse
xtemplate	zhnumber			

附录 B matlab 源程序

2.1 问题一相关代码

```

function [D, V, flag, AB] = position_diameter(S, theta)
%POSITION_DIAMETER 问题1: 交会定位区域直径 (向量化暴力求直径)
% [D, V, flag, AB] = position_diameter(S, theta)
%
% 输入 (两项, 均为用户输入):
%     S      - n×2 矩阵, 每行是一个检测点坐标 (x,y), 单位: 米
%     theta  - n×1 向量, 各检测点处测得的示向度 (方位角, 单位: 度, 范围 [0,360))
%
% 输出:
%     D      - 定位区域 (凸多边形) 的直径, 单位: 米
%     V      - 定位区域顶点 (逆时针排列, m×2 矩阵)
%     flag    - 状态: 1=区域有界 (直径有效), 0=区域无界 (直径无意义)
%     AB     - 直径两 endpoints (2×2 矩阵), 可选输出
%
% 方法 (对应问题一的解法):
%     每个检测点 S_i 确定一个张角 2*tau 的视界楔形
%     W_i = { P : |wrap(arg(P - S_i) - theta_i)| <= tau }
%     定位区域 R = 所有楔形之交, 是一个凸多边形;
%     其直径 = 凸多边形顶点对距离的最大值 (向量化暴力法, 无循环求得)。
tau = 1;      % 测向误差界 (度)
M = 10000;    % 裁剪用包围盒半边长 (米), 可自行调整
n = size(S, 1);
if ~(n >= 1 && size(S,2) == 2 && numel(theta) == n)
    error('输入尺寸不匹配: S 应为 n×2, theta 应为 n×1');
end
% ---- 0. 有界性判定 (问题一: 最小包围弧 > 2*tau 才有界) ----
[flag, arc] = boundedness(theta, tau);
if ~flag
    warning('示向度最小包围弧 = %.2f° <= 2*tau = %.0f°, 定位区域无界, 直径无意义.', arc, 2*tau);
end
% ---- 1. 用包围盒初始化多边形 ----
V = [-M -M; M -M; M M; -M M];
% ---- 2. 逐个楔形裁剪 (每个楔形 = 两条半平面 n'*P <= c) ----
for i = 1:n
    [n1, c1, n2, c2] = wedge_halfplanes(S(i,:), theta(i), tau);
    V = clip_halfplane(V, n1, c1);
    V = clip_halfplane(V, n2, c2);
end
if isempty(V)
    flag = 0; D = 0; V = []; AB = []; return;
end
% ---- 3. 取凸包 (去掉共线/重复点, 得到规范凸多边形) ----
V = convex_hull(V);
% ---- 4. 向量化暴力求直径 (顶点对最大距离, 无循环) ----
[D, AB] = polygon_diameter(V);
% ---- 5. 若区域触及包围盒, 说明实际无界 ----

```

```

if flag && (max(abs(V(:))) > 0.99 * M)
flag = 0;
warning('区域触及包围盒边界，可能无界，直径结果不可靠。');
end
end
%% ===== 子函数 =====
function [n1, c1, n2, c2] = wedge_halfplanes(S, th, tau)
% 楔形 W(S,th,tau) 的两条边界半平面 n'*P <= c
% 边界1: 方向角 th-tau 的射线 (外法向 n1)
% 边界2: 方向角 th+tau 的射线 (外法向 n2)
a1 = (th - tau) * pi / 180;
a2 = (th + tau) * pi / 180;
n1 = [sin(a1), -cos(a1)]; % 边界 th-tau 的外法向
c1 = n1 * S(:);
n2 = [-sin(a2), cos(a2)]; % 边界 th+tau 的外法向
c2 = n2 * S(:);
end
function P = clip_halfplane(P, n, c)
% Sutherland-Hodgman 半平面裁剪: 保留 n'*p <= c
if isempty(P), return; end
m = size(P, 1);
Q = zeros(0, 2);
for i = 1:m
j = mod(i, m) + 1;
pi_ = P(i, :);
pj = P(j, :);
fi = n * pi_ - c;
fj = n * pj - c;
if fi <= 0
Q(end+1, :) = pi_; %ok<AGROW>
end
if (fi <= 0) ~= (fj <= 0)
t = fi / (fi - fj);
Q(end+1, :) = pi_ + t * (pj - pi_); %ok<AGROW>
end
end
P = Q;
end
function H = convex_hull(P)
% 单调链凸包 (返回逆时针顶点)
if size(P, 1) <= 2, H = P; return; end
P = unique(P, 'rows');
P = sortrows(P, [1 2]);
lower = zeros(0, 2);
for i = 1:size(P, 1)
while size(lower, 1) >= 2 && ...
cross2(lower(end,:) - lower(end-1,:), P(i,:) - lower(end-1,:)) <= 0

```

```

lower(end, :) = [];
end
lower(end+1, :) = P(i, :);      %#ok<AGROW>
end
upper = zeros(0, 2);
for i = size(P, 1):-1:1
while size(upper, 1) >= 2 && ...
cross2(upper(end,:) - upper(end-1,:), P(i,:) - upper(end-1,:)) <= 0
upper(end, :) = [];
end
upper(end+1, :) = P(i, :);      %#ok<AGROW>
end
H = [lower(1:end-1, :); upper(1:end-1, :)];
end
function [D, AB] = polygon_diameter(H)
%POLYGON_DIAMETER 向量化暴力法求凸多边形直径（无循环）
% 输入 H: 逆时针凸多边形顶点 (m×2)
% 输出 D: 直径（米）；AB: 直径两 endpoints (2×2)
%
% 原理：凸多边形直径 = 顶点对距离的最大值（问题一定理）。
% 顶点数 m 很小，直接用隐式扩展一次性算出所有顶点对距离，全程无循环。
m = size(H, 1);
if m < 2
D = 0; AB = zeros(0, 2); return;
end
% 所有顶点对的坐标差：m×m（利用 MATLAB 隐式扩展，无需双重循环）
dx = H(:,1) - H(:,1).'; % x_i - x_j
dy = H(:,2) - H(:,2).'; % y_i - y_j
D2 = dx.^2 + dy.^2; % 所有顶点对平方距离
[D2max, idx] = max(D2(:)); % 最大平方距离及其线性索引
[i, j] = ind2sub([m m], idx); % 对应顶点对
D = sqrt(D2max);
AB = [H(i,:); H(j,:)];
end
function [flag, arc] = boundedness(theta, tau)
% 有界性判定：最小包围弧 > 2*tau 则定位区域有界
th = sort(mod(theta(:), 360));
n = numel(th);
if n < 2
flag = false; arc = 360; return;
end
gmax = 0;
for i = 1:n
gap = mod(th(mod(i,n)+1) - th(i), 360);
gmax = max(gmax, gap);
end
if gmax == 0

```

```

arc = 0;    % 所有示向度相同 -> 退化为单锥，无界
else
arc = 360 - gmax;
end
flag = arc > 2*tau;
end
function z = cross2(a, b)
z = a(1)*b(2) - a(2)*b(1);
end

```

% draw_counterexample.m

% 绘制等边三角形反例：以三角形直径为直径的圆无法覆盖该三角形，而最小覆盖圆的半径更大。

% 生成无标题的图像。

```

function draw_counterexample()
    root = fileparts(mfilename('fullpath'));
    cd(root);

    % 等边三角形顶点，边长 D = 1
    D = 1;
    % 顶点
    V1 = [0; 0];
    V2 = [D; 0];
    V3 = [D/2; D*sqrt(3)/2];

    % 最小覆盖圆（外接圆）
    % 中心 C
    C = [D/2; D*sqrt(3)/6];
    % 半径 R_MEC = D/sqrt(3)
    R_MEC = D/sqrt(3);

    % 直径为 D 的圆（设圆心也为C，这样最能覆盖更多的点）
    R_D = D/2;

    f = figure('Color', 'w', 'Position', [200, 200, 600, 600], 'Visible', 'off');
    hold on;
    axis equal;

    % 画等边三角形
    fill([V1(1), V2(1), V3(1)], [V1(2), V2(2), V3(2)], [0.9 0.9 0.9], 'EdgeColor', 'k',
        'LineWidth', 1.5, 'DisplayName', '等边三角形区域 K');

    % 画最小覆盖圆
    theta = linspace(0, 2*pi, 200);
    plot(C(1) + R_MEC*cos(theta), C(2) + R_MEC*sin(theta), 'b-', 'LineWidth', 2, 'DisplayName',
        '最小覆盖圆（半径 D/\surd3）');

```

```

% 画直径为 D 的圆
plot(C(1) + R_D*cos(theta), C(2) + R_D*sin(theta), 'r--', 'LineWidth', 2, 'DisplayName',
     '直径为 D 的圆 (半径 D/2)');

% 画出三角形的顶点
plot([V1(1), V2(1), V3(1)], [V1(2), V2(2), V3(2)], 'k.', 'MarkerSize', 15,
     'HandleVisibility', 'off');

% 标注中心点
plot(C(1), C(2), 'k+', 'MarkerSize', 8, 'HandleVisibility', 'off');

% 图例和坐标系设置
legend('Location', 'southoutside', 'FontSize', 12);
axis off;

% 保存图像
save_path = fullfile(root, 'counterexample_triangle.png');
exportgraphics(f, save_path, 'Resolution', 300, 'BackgroundColor', 'white');
close(f);
fprintf('反例图像已保存至: %s\n', save_path);
end

```

```

%% 示例：问题1 定位区域直径计算
% 输入项（两项都要用户自行输入）：
%   S      - 检测点坐标 (x,y)，每行一个点，单位：米
%   theta  - 各检测点的示向度（方位角），单位：度，范围 [0,360)
clc; clear;

% ===== 自行修改：检测点坐标 与 示向度 =====
% 每一行是一个检测点 (x, y)，对应 theta 里同一行的示向度（度）
% 想增加/减少检测点，只需同步增删 S 与 theta 的行数

% --- 示例1：两个检测点 ---
S = [ 0, 0;      % 检测点1 坐标
      1000, 0 ]; % 检测点2 坐标
theta = [90; 135]; % 检测点1、2 的示向度（度）

% --- 示例2：三个检测点（取消注释即可用） ---
% S = [ 0, 0;
%       1000, 0;
%       0, 800 ];
% theta = [90; 135; 30];

% --- 示例3：交互式输入（取消注释即可用） ---
% n = input('请输入检测点个数 n = ');

```

```

% S = zeros(n, 2); theta = zeros(n, 1);
% for i = 1:n
%     fprintf('检测点 %d:\n', i);
%     S(i,:) = input(' 坐标 [x, y] (米) = ');
%     theta(i) = input(' 示向度 (度 [0,360)) = ');
% end

% =====

[D, V, flag, AB] = position_diameter(S, theta);

fprintf('=====\n');
fprintf('定位区域直径 D = %.4f 米\n', D);
fprintf('有界性 flag = %d (1=有界, 0=无界) \n', flag);
fprintf('定位区域顶点 (逆时针, m×2) :\n');
disp(V);
fprintf('直径两端点 AB (旋转卡壳求出) :\n');
disp(AB);
fprintf('=====\n');

% 可视化 (可选)
figure; hold on; axis equal;
if ~isempty(V)
    plot([V(:,1); V(1,1)], [V(:,2); V(1,2)], 'k-', 'LineWidth', 1.5); % 定位区域边界
    fill(V(:,1), V(:,2), [0.9 0.95 1]); % 填充
end
plot(S(:,1), S(:,2), 'ro', 'MarkerFaceColor', 'r', 'MarkerSize', 7); % 检测点
for i = 1:size(S,1)
    text(S(i,1)+20, S(i,2)+20, sprintf('S%d', i));
    u = [cosd(theta(i)), sind(theta(i))];
    quiver(S(i,1), S(i,2), 300*u(1), 300*u(2), 0, 'r--', 'LineWidth', 1);
end
% 画出直径线段 (旋转卡壳求出的两个端点连线)
if ~isempty(AB) && size(AB,1) == 2
    plot(AB(:,1), AB(:,2), 'b-', 'LineWidth', 2.5);
    plot(AB(:,1), AB(:,2), 'bs', 'MarkerFaceColor', 'b', 'MarkerSize', 8);
end
title(sprintf('定位区域直径 D = %.2f m (蓝线为直径)', D));
grid on;

```

```

function D = diameter_many(V)
%DIAMETER_MANY 批量计算多个凸多边形的直径 (向量化, R2016b+ 隐式扩展)
% D = diameter_many(V)
%
% 输入 V: 1×P cell 数组, V{p} 为第 p 个多边形的顶点 (m_p×2, 逆时针)
% 输出 D: P×1 各多边形的直径 (米)

```

```

%
% 原理：凸多边形直径 = 顶点对最大距离。
% 顶点少、多边形多时，瓶颈是逐多边形循环；本函数把所有多边形堆成三维数组，
% 用隐式扩展一次性算完所有顶点对距离，全程向量化（仅堆叠一步有循环）。

P = numel(V);
mmax = max(cellfun(@size, V, 1), V);

% 堆成 mmax×2×P 三维数组，缺省顶点用 NaN 填充
X = NaN(mmax, 2, P);
for p = 1:P
    X(1:size(V{p},1), :, p) = V{p};
end

% 所有顶点对 (i,j) 的坐标差：mmax×mmax×P（隐式扩展，无双重循环）
dx = X(:,1,:) - permute(X(:,1,:), [2 1 3]); % x_i - x_j
dy = X(:,2,:) - permute(X(:,2,:), [2 1 3]); % y_i - y_j
D2 = dx.^2 + dy.^2; % 所有顶点对平方距离

D2(isnan(D2)) = 0; % NaN（补位处）置 0，不影响取最大值

D = sqrt(max(D2, [], [1 2])); % 1×1×P
D = squeeze(D); % P×1
end

```

2.2 问题二相关代码

```

function [out, V, S] = main_problem2(mode)
%MAIN_PROBLEM2 默认仅搜索、绘图与导出；'verify' 单独验证，'full' 另做敏感性分析。
if nargin == 0, mode = 'run'; end
assert(any(strcmp(mode, {'run','verify','full'})), '模式应为 run、verify 或 full。');
root = fileparts(mfilename('fullpath'));
old = pwd; cleanup = onCleanup(@() cd(old));
cd(root); addpath(genpath(root));
cfg = config_problem2();
if ~isfolder(cfg.results_dir), mkdir(cfg.results_dir); end
if ~isfolder(cfg.fig_dir), mkdir(cfg.fig_dir); end
out = []; V = []; S = [];
t_all = tic;
fprintf('问题二：第二检测点选择 | MATLAB %s\n', version);
if any(strcmp(mode, {'verify','full'}))
    V = verify_problem2();
    assert(V.nfail == 0, '验证失败，请检查 results/verify_results.mat。');
    if strcmp(mode, 'verify'), return; end
end
end

```

```

out = outer_search(cfg);
if strcmp(mode, 'full'), S = sensitivity(out, cfg); end
plot_problem2(cfg, out);
out.t_run = toc(t_all);
export_problem2(out, cfg);
save(fullfile(cfg.results_dir, 'out.mat'), 'out', 'cfg', 'V', 'S', '-v7');
fprintf('完成: 搜索 %.2f s, 总计 %.2f s (均不含 MATLAB 启动) 。\n', out.t_total, out.t_run);
fprintf('数值及论文数据: results/; 图: figures/。 \n');
end

```

```

function problem2_discrete_sweep()
% PROBLEM2_DISCRETE_SWEEP 基于 Kershner 定理和集合覆盖规划离散检测点及 TSP 最短路

R = 56.5; % 最坏误差半径
r = 20; % 探测半径
v = 5; % 移动速度 5m/s (假设)
t_measure = 5; % 每次测量耗时 5s

fprintf('=== 问题2: 离散检测点全覆盖路径规划 ===\n');
fprintf('误差半径 R = %.1f m, 探测半径 r = %.1f m\n', R, r);

% 1. 离散化被覆盖区域 D(0, R)
[X, Y] = meshgrid(linspace(-R, R, 150));
mask = X.^2 + Y.^2 <= R^2;
Px = X(mask); Py = Y(mask);
N_target = length(Px);

% 2. 基于 Kershner 正六边形密铺定理生成候选中心
dx = r * sqrt(3) / 2 * 0.8;
dy = r * 3/4 * 0.8;
[CX, CY] = meshgrid(-R-r:dx:R+r, -R-r:dy:R+r);
shift = repmat([0, dx/2], size(CX,1), ceil(size(CX,2)/2));
shift = shift(:, 1:size(CX,2));
CX = CX + shift;

c_mask = CX.^2 + CY.^2 <= (R+r/4)^2;
Cx = CX(c_mask); Cy = CY(c_mask);
N_cand = length(Cx);

% 3. 构造覆盖矩阵 A
A = zeros(N_target, N_cand);
for j = 1:N_cand
    dist2 = (Px - Cx(j)).^2 + (Py - Cy(j)).^2;
    A(:, j) = (dist2 <= r^2);
end

```

```

% 4. 求解最小集合覆盖问题 (Set Cover ILP)
f = ones(N_cand, 1);
intcon = 1:N_cand;
b = -ones(N_target, 1);

options = optimoptions('intlinprog', 'Display', 'off');
[x_opt, fval, exitflag] = intlinprog(f, intcon, -A, b, [], [], zeros(N_cand,1),
    ones(N_cand,1), options);

if exitflag > 0
    N_opt = round(fval);
    sel = x_opt > 0.5;
    Sel_X = Cx(sel); Sel_Y = Cy(sel);
    fprintf('成功求得最小检测点数量: %d 个\n', N_opt);

    pts = [Sel_X, Sel_Y];

% 5. 求解 TSP 遍历最短路径 (贪心 + 2-opt)
start_pt = [0, 0];
[tour_idx, tour_dist] = solve_tsp(pts, start_pt);

total_time = N_opt * t_measure + tour_dist / v;
fprintf('TSP 最短行走距离: %.2f m\n', tour_dist);
fprintf('预估总耗时 (含 %d 次停机检测): %.2f s\n', N_opt, total_time);

% 6. 可视化绘图
plot_discrete_coverage(R, r, pts, tour_idx, start_pt, tour_dist, N_opt);
else
    fprintf('ILP 求解失败, 无法找到全覆盖子集.\n');
end
end

function [best_tour, best_dist] = solve_tsp(pts, start_pt)
    N = size(pts, 1);
    unvisited = 1:N;
    curr = start_pt;
    tour = [];
    dist = 0;

% 贪心初始化
while ~isempty(unvisited)
    [~, idx] = min(sum((pts(unvisited,:) - curr).^2, 2));
    next = unvisited(idx);
    dist = dist + norm(pts(next,:) - curr);
    tour(end+1) = next;
    curr = pts(next,:);
    unvisited(idx) = [];
end

```

```

end

% 简单的 2-opt 优化
improved = true;
while improved
    improved = false;
    for i = 1:N-2
        for j = i+2:N
            new_tour = tour;
            new_tour(i+1:j) = wrev(tour(i+1:j)); % 逆序子路径
            new_dist = calc_tour_dist(new_tour, pts, start_pt);
            if new_dist < dist - 1e-5
                tour = new_tour;
                dist = new_dist;
                improved = true;
            end
        end
    end
    best_tour = tour;
    best_dist = dist;
end

function rev = wrev(arr)
    rev = arr(end:-1:1);
end

function d = calc_tour_dist(tour, pts, start_pt)
    d = norm(pts(tour(1),:) - start_pt);
    for i = 1:length(tour)-1
        d = d + norm(pts(tour(i+1),:) - pts(tour(i),:));
    end
end

function plot_discrete_coverage(R, r, pts, tour_idx, start_pt, tour_dist, N_opt)
    fig = figure('Color', 'w', 'Position', [150, 150, 750, 700]);
    hold on; axis equal;

    % 画目标误差圆
    theta = linspace(0, 2*pi, 200);
    plot(R*cos(theta), R*sin(theta), 'r--', 'LineWidth', 2, 'DisplayName', sprintf('最坏误差圆 (R=%.1fm)', R));

    % 画起点
    plot(start_pt(1), start_pt(2), 'kp', 'MarkerFaceColor', 'y', 'MarkerSize', 15,
        'DisplayName', '假定第二观测点(起点)');

```

```

% 画检测点的探测范围
h_circle = gobjects(0);
for i = 1:N_opt
    h = fill(pts(i,1)+r*cos(theta), pts(i,2)+r*sin(theta), [0.3 0.6 0.9], 'EdgeColor', 'b',
            'FaceAlpha', 0.2, 'HandleVisibility', 'off');
    if i==1, h_circle = h; end
end
set(h_circle, 'DisplayName', '20m 探测覆盖区');

% 画 TSP 路径
path_pts = [start_pt; pts(tour_idx, :)];
plot(path_pts(:,1), path_pts(:,2), 'm-', 'LineWidth', 2.5, 'DisplayName',
      '无人狗遍历检测路线');

% 标记检测点序号
plot(pts(:,1), pts(:,2), 'mo', 'MarkerFaceColor', 'm', 'MarkerSize', 8, 'HandleVisibility',
      'off');
for i = 1:N_opt
    text(pts(tour_idx(i),1)+2, pts(tour_idx(i),2)+2, num2str(i), 'Color', 'k', 'FontWeight',
          'bold', 'FontSize', 12);
end

title(sprintf('基于正六边形密铺的最优离散检测点全覆盖路径\n(最少停机检测点: %d 个, 路线长度:
              %.2f m)', N_opt, tour_dist), 'FontSize', 13);
xlabel('X 坐标 (m)'); ylabel('Y 坐标 (m)');
legend('Location', 'northeastoutside', 'FontSize', 10);
grid on; xlim([-R-r-5, R+r+5]); ylim([-R-r-5, R+r+5]);

out_file = fullfile(fileparts(mfilename('fullpath')), 'results',
                    'problem2_discrete_coverage.png');
saveas(fig, out_file);
fprintf('绘图成功! 已保存至: %s\n', out_file);
end

```

```

function problem2_sweep_path()
% PROBLEM2_SWEEP_PATH 自适应计算并证明任意误差半径 R 下的最短全覆盖路径
% 不仅输出最坏情况 (R=56.5m) 的覆盖图, 还生成 R 属于 [21, 56.5] 时的最优路径长度曲线。

r = 20; % 仪器的探测半径

% =====
% 1. 计算不同 R 下的最优路径长度 (证明普适性)
% =====
fprintf('正在验证任意 R 属于 (20, 56.5] 的自适应最优全覆盖路径...\n');
R_array = linspace(21, 56.5, 30);
L_array = zeros(size(R_array));

```

```

for i = 1:length(R_array)
    R = R_array(i);
    [L_array(i), ~] = optimize_sweep(R, r, false);
end

% 绘制 L - R 关系曲线
fig1 = figure('Color', 'w', 'Position', [100, 100, 700, 500]);
plot(R_array, L_array, 'b-', 'LineWidth', 2.5);
hold on;
% 标注分界点
xline(40, 'r--', 'LineWidth', 1.5, 'Label', '拓扑相变点 (R=40m)');
plot(R_array, L_array, 'bo', 'MarkerFaceColor', 'w', 'MarkerSize', 4);

title('最短全覆盖路径长度 L 随误差圆半径 R 的变化规律', 'FontSize', 14);
xlabel('未知误差圆半径 R (m)', 'FontSize', 12);
ylabel('最优全覆盖路径长度 L (m)', 'FontSize', 12);
grid on;

text(25, 250, '2趟扫线 (U型拓扑)', 'FontSize', 12, 'Color', 'k');
text(45, 250, '3趟扫线 (S型拓扑)', 'FontSize', 12, 'Color', 'k');

out_file1 = fullfile(fileparts(mfilename('fullpath')), 'results', 'problem2_LR_curve.png');
if ~isfolder(fileparts(out_file1)), mkdir(fileparts(out_file1)); end
saveas(fig1, out_file1);
fprintf('规律证明曲线已保存至: %s\n', out_file1);

% =====
% 2. 针对最坏情况 (R = 56.5) 生成具体清扫路线图
% =====
R_worst = 56.5;
[L_worst, pts_worst] = optimize_sweep(R_worst, r, true);

fprintf('\n==== 优化完成 =====\n');
fprintf('最坏情况 (R=%1f m) 下的最短全覆盖路径总长度: %.2f m\n', R_worst, L_worst);

plot_sweep_result(pts_worst, R_worst, r, L_worst);
end

function [fval, pts] = optimize_sweep(R, r, verbose)
% 生成高密度测试点
[X, Y] = meshgrid(linspace(-R, R, 150));
mask = X.^2 + Y.^2 <= R^2;
Px = X(mask);
Py = Y(mask);

options = optimoptions('fmincon', 'Display', 'none', 'Algorithm', 'sqp', ...

```

```

'MaxFunctionEvaluations', 2000, 'StepTolerance', 1e-6);

% 根据 R 动态选择扫线拓扑
if R <= 40
    % 2 趟扫线 (U型拓扑)
    x0 = [R, R/2];
    lb = [0, 0];
    ub = [R, R];
    obj = @(v) 4*v(1) + 2*v(2);
    nonlcon = @(v) constraint_2sweep(v, Px, Py, r);
else
    % 3 趟扫线 (S型/割草机拓扑)
    x0 = [R*0.8, R*0.6, R*0.8];
    lb = [0, 0, 0];
    ub = [R, R, R];
    obj = @(v) 4*v(1) + 2*v(3) + 2*sqrt((v(3)-v(1))^2 + v(2)^2);
    nonlcon = @(v) constraint_3sweep(v, Px, Py, r);
end

if verbose, fprintf('开始寻优 R = %.1f m ...\n', R); end
[v_opt, fval] = fmincon(obj, x0, [], [], [], [], lb, ub, nonlcon, options);

% 重建节点
if R <= 40
    x1 = v_opt(1); y1 = v_opt(2);
    pts = [x1, y1; -x1, y1; -x1, -y1; x1, -y1];
else
    x1 = v_opt(1); y1 = v_opt(2); x2 = v_opt(3);
    pts = [x1, y1; -x1, y1; -x2, 0; x2, 0; x1, -y1; -x1, -y1];
end
end

function [c, ceq] = constraint_2sweep(v, Px, Py, r)
    x1 = v(1); y1 = v(2); r2 = r^2;
    d1 = dist2seg(Px, Py, x1, y1, -x1, y1);
    d2 = dist2seg(Px, Py, -x1, y1, -x1, -y1);
    d3 = dist2seg(Px, Py, -x1, -y1, x1, -y1);
    min_d = min(cat(2, d1, d2, d3), [], 2);
    c = min_d - r2 + 1e-4;
    ceq = [];
end

function [c, ceq] = constraint_3sweep(v, Px, Py, r)
    x1 = v(1); y1 = v(2); x2 = v(3); r2 = r^2;
    d1 = dist2seg(Px, Py, x1, y1, -x1, y1);
    d2 = dist2seg(Px, Py, -x1, y1, -x2, 0);
    d3 = dist2seg(Px, Py, -x2, 0, x2, 0);

```

```

d4 = dist2seg(Px, Py, x2, 0, x1, -y1);
d5 = dist2seg(Px, Py, x1, -y1, -x1, -y1);
min_d = min(cat(2, d1, d2, d3, d4, d5), [], 2);
c = min_d - r2 + 1e-4;
ceq = [];
end

function d2 = dist2seg(px, py, ax, ay, bx, by)
dx = bx - ax; dy = by - ay; l2 = dx^2 + dy^2;
if l2 == 0
    d2 = (px - ax).^2 + (py - ay).^2; return;
end
t = max(0, min(1, ((px - ax)*dx + (py - ay)*dy) / l2));
projx = ax + t * dx; projy = ay + t * dy;
d2 = (px - projx).^2 + (py - projy).^2;
end

function plot_sweep_result(pts, R, r, fval)
fig = figure('Color', 'w', 'Position', [150, 150, 750, 650]);
hold on; axis equal;

n_pts = size(pts, 1);
h_fill = gobjects(0);
for i=1:(n_pts-1)
    h = plot_capsule(pts(i,1), pts(i,2), pts(i+1,1), pts(i+1,2), r);
    if i==1, h_fill = h; end
end
set(h_fill, 'DisplayName', '20m 探测器扫掠覆盖区');

theta = linspace(0, 2*pi, 200);
plot(R*cos(theta), R*sin(theta), 'r--', 'LineWidth', 2, 'DisplayName',
    sprintf('未知干扰源可能区域 (R=%.1fm)', R));
plot(0, 0, 'rx', 'MarkerSize', 10, 'LineWidth', 2, 'DisplayName', '最坏误差圆心');
plot(pts(:,1), pts(:,2), 'b-', 'LineWidth', 2.5, 'DisplayName', '无人狗最短覆盖路径');
plot(pts(:,1), pts(:,2), 'bo', 'MarkerFaceColor', 'w', 'MarkerSize', 5, 'HandleVisibility',
    'off');
plot(pts(1,1), pts(1,2), 'gp', 'MarkerFaceColor', 'g', 'MarkerSize', 15, 'DisplayName',
    '路径起点');
plot(pts(end,1), pts(end,2), 'ks', 'MarkerFaceColor', 'k', 'MarkerSize', 10, 'DisplayName',
    '路径终点');

title(sprintf('最坏情况 (R=%.1fm) 的最短全覆盖扫雷路径 (L=%.2fm)', R, fval), 'FontSize', 13);
xlabel('X 坐标 (m)'); ylabel('Y 坐标 (m)');
legend('Location', 'northeastoutside', 'FontSize', 10);
grid on; xlim([-R-r-5, R+r+5]); ylim([-R-r-5, R+r+5]);

out_file = fullfile(fileparts(mfilename('fullpath')), 'results',

```

```

        'problem2_coverage_path.png');
saveas(fig, out_file);
fprintf('覆盖区域绘图成功! 已保存至: %s\n', out_file);
end

function h = plot_capsule(ax, ay, bx, by, r)
    dir = [bx-ax, by-ay]; len = norm(dir);
    if len == 0
        theta = linspace(0, 2*pi, 50);
        h = fill(ax+r*cos(theta), ay+r*sin(theta), [0.85 0.93 1], 'EdgeColor', 'none',
            'FaceAlpha', 0.6, 'HandleVisibility', 'off');
        return;
    end
    dir = dir / len; perp = [-dir(2), dir(1)];
    p1 = [ax, ay] + r*perp; p2 = [bx, by] + r*perp;
    p3 = [bx, by] - r*perp; p4 = [ax, ay] - r*perp;
    h = fill([p1(1) p2(1) p3(1) p4(1)], [p1(2) p2(2) p3(2) p4(2)], [0.85 0.93 1], 'EdgeColor',
        'none', 'FaceAlpha', 0.6, 'HandleVisibility', 'off');
    theta = linspace(0, 2*pi, 30);
    fill(ax+r*cos(theta), ay+r*sin(theta), [0.85 0.93 1], 'EdgeColor', 'none', 'FaceAlpha',
        0.6, 'HandleVisibility', 'off');
    fill(bx+r*cos(theta), by+r*sin(theta), [0.85 0.93 1], 'EdgeColor', 'none', 'FaceAlpha',
        0.6, 'HandleVisibility', 'off');
end

```

```

% get_continuous_region.m
% 读取 results/out.mat, 利用真实的评价函数 eval_P_bounds 在候选点附近进行密集网格采样,
% 从而找出严格满足精度容差 ( $U(P) \leq m\_hat + \epsilon$ ) 的连续可行区域, 并绘制热力图。
%
    输出两张图: 局部图 (fig4_continuous_region_local.png) 和全局图 (fig4_continuous_region_global.png)

function get_continuous_region()
    root = fileparts(mfilename('fullpath'));
    old = pwd; cleanup = onCleanup(@() cd(old));
    cd(root); addpath(genpath(root));

    fprintf('正在加载结果数据...\n');
    load('results/out.mat', 'out', 'cfg');

    P_fines = [out.fines.P];
    Q = P_fines(:, out.near_indices);

    % 目标阈值
    threshold = out.m_hat + cfg.tau_m;
    fprintf('真实的最优上界基准 m_hat = %.4f m\n', out.m_hat);
    fprintf('寻找满足  $U(P) \leq %.4f m$  ( $m\_hat + \%.1f$ ) 的连续区域\n', threshold, cfg.tau_m);

```

```

cfg.kill_threshold = threshold;
cfg.eval_tol = cfg.tol_fine;
cfg.eval_budget = 400;

[Eb,~] = E_polygon(cfg);

% ===== 1. 局部图计算 =====
if out.P_f2(2) > 0
    Q_local = Q(:, Q(2,:) > 0);
else
    Q_local = Q(:, Q(2,:) < 0);
end

margin_l = 15;
min_x_l = min(Q_local(1,:)) - margin_l; max_x_l = max(Q_local(1,:)) + margin_l;
min_y_l = min(Q_local(2,:)) - margin_l; max_y_l = max(Q_local(2,:)) + margin_l;

grid_size_l = 50;
x_vec_l = linspace(min_x_l, max_x_l, grid_size_l);
y_vec_l = linspace(min_y_l, max_y_l, grid_size_l);
[X_l, Y_l] = meshgrid(x_vec_l, y_vec_l);
Z_U_l = NaN(size(X_l));

fprintf('开始对局部的 %dx%d 网格进行严密计算...\n', grid_size_l, grid_size_l);
tic;
for i = 1:numel(X_l)
    P_test = [X_l(i); Y_l(i)];
    if ~inpolygon(P_test(1), P_test(2), Eb(1,:), Eb(2:)), continue; end
    res = eval_P_bounds(P_test, cfg);
    Z_U_l(i) = res.U;
end
fprintf('局部网格计算完成, 耗时 %.2f 秒.\n', toc);

% ===== 2. 全局图计算 =====
margin_g = 15;
min_x_g = min(Eb(1,:)) - margin_g; max_x_g = max(Eb(1,:)) + margin_g;
min_y_g = min(Eb(2,:)) - margin_g; max_y_g = max(Eb(2,:)) + margin_g;

grid_size_g = 60;
x_vec_g = linspace(min_x_g, max_x_g, grid_size_g);
y_vec_g = linspace(min_y_g, max_y_g, grid_size_g);
[X_g, Y_g] = meshgrid(x_vec_g, y_vec_g);
Z_U_g = NaN(size(X_g));

fprintf('开始对全局的 %dx%d 网格进行严密计算...\n', grid_size_g, grid_size_g);
tic;

```

```

for i = 1:numel(X_g)
    P_test = [X_g(i); Y_g(i)];
    if ~inpolygon(P_test(1), P_test(2), Eb(1,:), Eb(2,:)), continue; end
    res = eval_P_bounds(P_test, cfg);
    Z_U_g(i) = res.U;
end
fprintf('全局网格计算完成, 耗时 %.2f 秒.\n', toc);

levels = linspace(out.m_hat, threshold, 6);

% ===== 绘制局部图 =====
f1 = figure('Color', 'w', 'Position', [100 100 850 650], 'Visible', 'off');
hold on;
contourf(X_l, Y_l, Z_U_l, levels, 'LineStyle', 'none');
colormap(flipud(parula));
colorbar('Ticks', levels, 'TickLabels', arrayfun(@(x) sprintf('%.2f', x), levels,
    'UniformOutput', false));
[~, h_cl] = contour(X_l, Y_l, Z_U_l, [threshold, threshold], 'LineColor', 'r', 'LineWidth',
    2, 'DisplayName', 'Optimal Region Boundary');
plot(Eb(1,:), Eb(2,:), 'k.', 'MarkerSize', 2, 'DisplayName', 'Region E Boundary');
scatter(Q(1,:), Q(2,:), 40, 'mo', 'filled', 'MarkerEdgeColor', 'w', 'DisplayName',
    'Discrete Candidates');
plot(out.P_f2(1), out.P_f2(2), 'wp', 'MarkerSize', 14, 'MarkerFaceColor', 'r',
    'DisplayName', 'Selected Point P');
axis equal; grid on;
xlim([min_x_l, max_x_l]); ylim([min_y_l, max_y_l]);
xlabel('x (m)'); ylabel('y (m)');
legend('Location', 'best');

save_path_l = fullfile(cfg.fig_dir, 'fig4_continuous_region_local.png');
exportgraphics(f1, save_path_l, 'Resolution', 250, 'BackgroundColor', 'white');
close(f1);

% ===== 绘制全局图 =====
fg = figure('Color', 'w', 'Position', [200 200 850 650], 'Visible', 'off');
hold on;
contourf(X_g, Y_g, Z_U_g, levels, 'LineStyle', 'none');
colormap(flipud(parula));
colorbar('Ticks', levels, 'TickLabels', arrayfun(@(x) sprintf('%.2f', x), levels,
    'UniformOutput', false));
[~, h_cg] = contour(X_g, Y_g, Z_U_g, [threshold, threshold], 'LineColor', 'r', 'LineWidth',
    2, 'DisplayName', 'Optimal Region Boundary');
plot(Eb(1,:), Eb(2,:), 'k-', 'LineWidth', 1.5, 'DisplayName', 'Region E Boundary');
scatter(Q(1,:), Q(2,:), 20, 'mo', 'filled', 'MarkerEdgeColor', 'w', 'DisplayName',
    'Discrete Candidates');

plot(out.P_f2(1), out.P_f2(2), 'wp', 'MarkerSize', 14, 'MarkerFaceColor', 'r',

```

```

        'DisplayName', 'Selected Point P');
plot(out.P_f2(1), -out.P_f2(2), 'wp', 'MarkerSize', 14, 'MarkerFaceColor', 'r',
     'HandleVisibility', 'off');

axis equal; grid on;
xlim([min_x_g, max_x_g]); ylim([min_y_g, max_y_g]);
xlabel('x (m)'); ylabel('y (m)');
legend('Location', 'best');

save_path_g = fullfile(cfg.fig_dir, 'fig4_continuous_region_global.png');
exportgraphics(fg, save_path_g, 'Resolution', 250, 'BackgroundColor', 'white');
close(fg);

fprintf('已将全局和局部的连续区域分布图保存至 figures 目录\n');
end

```

2.3 问题三相关代码

```

function [comparison, summary] = problem3_compare_monte_carlo(trials, seed, scenario)
% 在完全相同的干扰源案例上，对比基础款和优化款问题三算法。
% [comparison,summary] = problem3_compare_monte_carlo(200,20260912,'uniform');
% scenario: uniform / edge / cluster / extreme。
% 两算法共享源数、频道、位置、接收半径及“位置-频道”固定误差场；
% 策略只接收模拟HTTP反馈，绝不读取场景真值。
if nargin<1, trials=200; end
if nargin<2, seed=20260912; end
if nargin<3, scenario='uniform'; end
validateattributes(trials,{'numeric'},{'scalar','integer','positive'});
assert(ismember(scenario,{'uniform','edge','cluster','extreme'}),'未知场景。');
root=fileparts(mfilename('fullpath')); addpath(root);
world=RandStream('mt19937ar','Seed',seed);
oldRng=rng; cleanup=onCleanup(@() rng(oldRng));
values=zeros(2*trials,13); algorithms=strings(2*trials,1); errors=strings(2*trials,1);
solvers=@problem3_solver,@problem3_solver_optimized; labels=["base" "optimized"];
for trial=1:trials
    n=randi(world,[10,16]); channels=randperm(world,20,n);
    theta=2*pi*rand(world,1,n); rho=1800*sqrt(rand(world,1,n));
    xy=[rho.*cos(theta);rho.*sin(theta)]; radius=1000+500*rand(world,1,n);
    if strcmp(scenario,'edge')
        xy=1800*[cos(theta);sin(theta)]; radius(:)=1000;
    elseif strcmp(scenario,'cluster')
        c=1400*[cos(theta(1));sin(theta(1))];
        xy=c+150*randn(world,2,n); xy=xy.*min(1,1800./vecnorm(xy));
    elseif strcmp(scenario,'extreme')
        xy(:,1)=[0;0]; xy(:,2)=[1800;0]; radius(:)=1000;
    end
end

```

```

end
for a=1:2
    rng(seed+trial); % 两算法的随机包围圆从同一状态开始
    [m,err]=runCase(solvers{a},channels,xy,radius,seed+104729*trial,scenario);
    row=2*(trial-1)+a;
    values(row,:)= [trial,n,m]; algorithms(row)=labels(a); errors(row)=err;
end
b=values(2*trial-1,5); o=values(2*trial,5);
fprintf('%4d/%d | N=%2d | 基础 %.1f s/源 | 优化 %.1f s/源 | 节省 %.1f s/源\n', ...
        trial,trials,n,b,o,b-o);
end
comparison=array2table(values,'VariableNames',{'Trial','N','Cleared','Total_s', ...
        'Average_s','Runtime_s','Complete','Move_s','Switch_s','Measure_s', ...
        'OpticalLaser_s','FinalCheck_s','AnchorRadius_m'});
comparison.Algorithm=algorithms; comparison.Error=errors;
comparison=movevars(comparison',{'Algorithm','Error'},'After','Trial');
b=comparison(comparison.Algorithm=="base",:);
o=comparison(comparison.Algorithm=="optimized",:);
summary=table(labels', [mean(b.Average_s);mean(o.Average_s)], ...
        [sum(b.Total_s)/sum(b.Cleared);sum(o.Total_s)/sum(o.Cleared)], ...
        [mean(b.Total_s);mean(o.Total_s)], [mean(b.Move_s);mean(o.Move_s)], ...
        [mean(b.Measure_s);mean(o.Measure_s)], [mean(b.Complete);mean(o.Complete)], ...
        [mean(b.Average_s<300);mean(o.Average_s<300)], ...
        'VariableNames',{'Algorithm','MeanAverage_s','PooledAverage_s','MeanTotal_s', ...
        'MeanMove_s','MeanMeasure_s','FullClearRate','Below300Rate'});
delta=b.Average_s-o.Average_s;
fprintf('\n%s场景: 基础 %.2f, 优化 %.2f s/源; 平均节省 %.2f s/源 (%.1f%) 。\n', ...
        scenario,mean(b.Average_s),mean(o.Average_s),mean(delta),100*mean(delta)/mean(b.Average_s));
fprintf('全清率: 基础 %.1f%%, 优化 %.1f%%; 低于300 s: 基础 %.1f%%, 优化 %.1f%%。 \n', ...
        100*mean(b.Complete),100*mean(o.Complete),100*mean(b.Average_s<300),100*mean(o.Average_s<300));
folder=fullfile(root,'问题3优化结果'); if ~isfolder(folder), mkdir(folder); end
prefix=fullfile(folder,['problem3_compare_' scenario]);
writetable(comparison,[prefix '_runs.csv']); writetable(summary,[prefix '_summary.csv']);
save([prefix '.mat'],'comparison','summary','seed','scenario');
f=figure('Color','w','Visible','off','Position',[100 100 900 650]);
tiledlayout(2,1);
nexttile; hb=plot(b.Average_s,'o-','DisplayName','基础款'); hold on;
ho=plot(o.Average_s,'o-','DisplayName','优化款');
yline(300,'r--','300 s','HandleVisibility','off');
ylabel('Time per source (s)'); legend([hb,ho],'Location','best');
title(sprintf('%s: paired Monte Carlo comparison',scenario)); grid on;
nexttile; bar(delta); yline(0,'k-'); xlabel('Paired trial');
ylabel('Base - optimized (s/source)'); grid on;
set(findall(f,'Type','axes'),'Color','w','XColor','k','YColor','k','GridColor',[.3 .3 .3]);
set(findall(f,'Type','text'),'Color','k');
exportgraphics(f,[prefix '.png'],'Resolution',150); close(f);
end

```

```

function [metrics,errText]=runCase(solver,channels,xy,radius,noiseSeed,scenario)
% 单案例封闭模拟器；真实数据不进入solver，仅通过四种接口结果反馈。
n=numel(channels); live=true(1,n); p=[0;0]; ch=1; t=0; parts=zeros(1,4);
clearTimes=[]; errText=""; result=struct(); start=tic;
try
    result=solver(@simulator);
catch err
    errText=string(err.message);
end
count=sum(~live); average=t/max(1,count); if count==0, average=inf; end
complete=~any(live) && isfield(result,'complete') && result.complete;
tail=nan; if ~isempty(clearTimes), tail=t-max(clearTimes); end
anchorRadius=nan;
if isfield(result,'anchorRadius'), anchorRadius=result.anchorRadius; end
assert(abs(t-sum(parts))<1e-6,'模拟计时分项不守恒。');
metrics=[count,t,average,toc(start),complete,parts,tail,anchorRadius];

function z=simulator(path,request)
    z=struct('accepted',true,'virtual_time_s',t);
    if strcmp(path,'/enter'), z.remaining_real_duration_s=1200; return; end
    if strcmp(path,'/exit'), z.exit_reason='user_exit'; return; end
    q=[request.position.x;request.position.y]; k=request.channel;
    dt=norm(q-p)/5; parts(1)=parts(1)+dt; t=t+dt; p=q;
    j=find(channels==k & live,1); d=inf;
    if ~isempty(j), d=norm(xy(:,j)-p); end
    if strcmp(path,'/measure')
        sw=double(ch~=k); ch=k; t=t+sw+5;
        parts(2)=parts(2)+sw; parts(3)=parts(3)+5;
        if isempty(j) || d>radius(j), z.measure_result='no_signal';
        elseif d<=5, z.measure_result='near';
        else
            % 确定性伪随机误差场：同一案例、频道、地点给两个算法相同误差。
            w=sin(q(1)*12.9898+q(2)*78.233+k*37.719+noiseSeed*.013)*43758.5453;
            e=2*(w-floor(w))-1;
            if strcmp(scenario,'extreme'), e=sign(e); end
            v=xy(:,j)-p; z.measure_result='direction';
            z.svd_deg=mod(round(atan2d(v(2),v(1))+e,2),360);
        end
    else
        ok=d<=20; dt=3+2*ok; t=t+dt; parts(4)=parts(4)+dt;
        if ok
            live(j)=false; z.clear_result='success'; clearTimes(end+1)=t;
        else
            z.clear_result='no_target_in_range';
        end
    end
end
end

```

```

        z.virtual_time_s=t;
    end
end

```

```

function [runs, sources] = problem3_monte_carlo(trials, seed, scenario)
% 蒙特卡洛仿真；仅调用 problem3_solver 的同一套策略，不复制或另写算法。
% [runs,sources] = problem3_monte_carlo(200,20260912);
% 压力场景：'edge'（边界、1000m接收）、'cluster'（聚集）、'extreme'（误差端点）。
% 默认：源数在10:16均匀取值，位置在圆内面积均匀，频道无放回抽样，
% 接收半径在[1000,1500]均匀。它们是本地实验假设，不是官方案例分布。
% 同地点同频道的误差固定；新地点误差独立均匀，且保留两位小数。
% 输出逐次总时间/平均清除时间，及逐源首次发现、清除时刻、清除间隔。
% Average_s = Total_s/Cleared，包含最后查漏；逐源清除间隔本身不含最后查漏。
if nargin<1, trials=200; end
if nargin<2, seed=20260912; end
if nargin<3, scenario='uniform'; end
validateattributes(trials,{ 'numeric' },{ 'scalar', 'integer', 'positive' });
assert(ismember(scenario,{ 'uniform', 'edge', 'cluster', 'extreme' }), '未知场景。');
root = fileparts(mfilename('fullpath'));
records = zeros(trials,11); sourceRows = zeros(0,6); errors = strings(trials,1);
% 单独的随机流确保包围圆工具内的 randperm 不会改变场景或测量噪声。
world = RandStream('mt19937ar', 'Seed', seed);
oldRng=rng; cleanup=onCleanup(@() rng(oldRng)); rng(seed); % 固定几何工具随机序
for trial=1:trials
    n=randi(world,[10,16]); channels=randperm(world,20,n);
    theta=2*pi*rand(world,1,n); rho=1800*sqrt(rand(world,1,n));
    xy=[rho.*cos(theta);rho.*sin(theta)]; radius=1000+500*rand(world,1,n);
    if strcmp(scenario,'edge')
        xy=1800*[cos(theta);sin(theta)]; radius(:)=1000;
    elseif strcmp(scenario,'cluster')
        c=1400*[cos(theta(1));sin(theta(1))];
        xy=c+150*randn(world,2,n);
        xy=xy.*min(1,1800./vecnorm(xy));
    elseif strcmp(scenario,'extreme')
        xy(:,1)=[0;0]; xy(:,2)=[1800;0]; radius(:)=1000;
    end
    live=true(1,n); p=[0;0]; ch=1; t=0; parts=zeros(1,4);
    noise=containers.Map('KeyType','char','ValueType','double');
    noiseRng=RandStream('mt19937ar','Seed',seed+trial);
    start=tic; complete=false; why=""; result=struct(); tail=nan;
    try
        result=problem3_solver(@simulator);
        complete=result.complete && ~any(live);
        ids=find(isfinite(result.clearTime));
        [times,order]=sort(result.clearTime(ids)); ids=ids(order);
        gaps=diff([0,times]);

```

```

        if ~isempty(times), tail=t-times(end); end % 最后一次清除后的查漏时间
        sourceRows=[sourceRows;[repmat(trial,numel(ids),1),ids', ...
            result.discoveryTime(ids)',times',gaps', ...
            (times-result.discoveryTime(ids))']]; %#ok<AGROW>
    catch err
        why=string(err.message); % 保留失败案例，不剔除后计算漂亮的均值
    end
    count=sum(~live); avg=t/max(1,count);
    if count==0, avg=inf; end
    records(trial,:)=n,count,t,avg,toc(start),complete,parts,tail; errors(trial)=why;
    fprintf('%4d/%d | %2d/%2d | 总 %.1f s | 平均 %.1f s | 全清 %d\n', ...
        trial,trials,count,n,t,avg,complete);
    assert(abs(t-sum(parts))<1e-6,'计时分项不守恒。');
end
runs=array2table(records,'VariableNames',{ 'N','Cleared','Total_s','Average_s', ...
    'Runtime_s','Complete','Move_s','Switch_s','Measure_s','OpticalLaser_s','FinalCheck_s'});
runs.Error=errors;
sources=array2table(sourceRows,'VariableNames',{ 'Trial','Channel','Discovery_s', ...
    'Clear_s','ClearInterval_s','DiscoveryToClear_s'});
folder=fullfile(root,'问题3结果'); if ~isfolder(folder), mkdir(folder); end
prefix=fullfile(folder,['problem3_mc_' scenario]);
writetable(runs,[prefix '_runs.csv']); writetable(sources,[prefix '_sources.csv']);
save([prefix '.mat'],'runs','sources','seed','scenario','result');
fprintf('\n全清率 %.1f%%; 逐次平均耗时的均值 %.2f s; 总时间/总清除数 %.2f s\n', ...
    100*mean(runs.Complete),mean(runs.Average_s),sum(runs.Total_s)/sum(runs.Cleared));
fprintf('同时满足全清且平均<300s: %.1f%%; 最慢一局平均 %.2f s\n', ...
    100*mean(runs.Complete & runs.Average_s<300),max(runs.Average_s));
f=figure('Color','w','Visible','off');
tiledlayout(2,1);
nexttile; plot(runs.Total_s,'o-'); ylabel('Total time (s)'); grid on;
nexttile; plot(runs.Average_s,'o-'); yline(300,'r--','300 s');
xlabel('Trial'); ylabel('Time per cleared source (s)'); grid on;
set(findall(f,'Type','axes'),'Color','w','XColor','k','YColor','k','GridColor',[.3 .3 .3]);
set(findall(f,'Type','text'),'Color','k');
exportgraphics(f,[prefix '.png'],'Resolution',150); close(f);

function z=simulator(path,request)
    % 真值仅封闭于此接口中；不向策略泄露个数、坐标、距离或接收半径。
    z=struct('accepted',true,'virtual_time_s',t);
    if strcmp(path,'/enter'), z.remaining_real_duration_s=1200; return; end
    if strcmp(path,'/exit'), z.exit_reason='user_exit'; return; end
    q=[request.position.x;request.position.y]; k=request.channel;
    dt=norm(q-p)/5; parts(1)=parts(1)+dt; t=t+dt; p=q;
    j=find(channels==k & live,1); d=inf;
    if ~isempty(j), d=norm(xy(:,j)-p); end
    if strcmp(path,'/measure')
        sw=double(ch~=k); ch=k; t=t+sw+5;
    end
end

```

```

parts(2)=parts(2)+sw; parts(3)=parts(3)+5;
if isempty(j) || d>radius(j), z.measure_result='no_signal';
elseif d<=5, z.measure_result='near';
else
    key=sprintf('%d:%.17g:%.17g',k,p(1),p(2));
    if ~isKey(noise,key)
        e=2*rand(noiseRng)-1;
        if strcmp(scenario,'extreme'), e=sign(e); end
        noise(key)=e;
    end
    z.measure_result='direction';
    v=xy(:,j)-p;
    z.svd_deg=mod(round(atan2d(v(2),v(1))+noise(key),2),360);
end
else
    ok=d<=20; dt=3+2*ok; t=t+dt; parts(4)=parts(4)+dt;
    if ok, live(j)=false; z.clear_result='success';
    else, z.clear_result='no_target_in_range'; end
end
z.virtual_time_s=t;
end
end

```

```

function out = problem3_solver(robotId, baseUrl)
% 问题三：覆盖排查 + 就近清除 + 自适应交会（仅适用于全向源）。
% 官方模拟器就绪后: out = problem3_solver('你们的参赛队号');
% 本地仿真由 problem3_monte_carlo 调用同一策略，策略不读取场景真值。
% 复用问题一 diameter_many, 问题二 wedge_planes/clip_halfplane/welzl_mec。
% 新增代码仅本文件与仿真文件；原问题一、二目录须保持在本文件旁边。
if nargin < 1, error('请传入参赛队号；本地测试请运行 problem3_monte_carlo。'); end
if nargin < 2, baseUrl = 'http://127.0.0.1:2026'; end
root = fileparts(mfilename('fullpath'));
addpath(fullfile(root,'问题1_交会定位'));
addpath(genpath(fullfile(root,'问题2_第二检测点选择')));
cfg = config_problem2();
local = isa(robotId,'function_handle');
if local, transport = robotId; robotId = 'local'; end
opt = weboptions('MediaType','application/json','Timeout',5);
pos = [0;0]; channel = 1; vt = 0; serial = 0; limit = inf; clock0 = tic;
known = false(1,20); done = known; regions = cell(1,20);
centers = nan(2,20); radii = inf(1,20); discovered = nan(1,20);
cleared = nan(1,20); history = cell(0,3);

% 75 m 方格的外接半径为 75/sqrt(2)。保留所有可能与目标圆相交的格子。
% 只有整个格子处于某次全频道扫描的 1000 m 接收圆内，才从待查表删除。
% 因而这里的网格给出连续区域覆盖证书，不是只检查离散点。

```

```

h = 75; margin = h/sqrt(2);
[gx,gy] = meshgrid(-1800:h:1800);
unseen = [gx(:)';gy(:)'];
unseen = unseen(:,vecnorm(unseen)<=1800+margin);
coverR = 1000-margin-1e-6;
% 原点加半径1500m上的正六边形顶点可覆盖这些带余量的格子。
% 外层格心 r<=1853.04, 距最近环站夹角<=30°, 最坏距离<934<coverR。
% 固定候选点避免把补查站选到边界外侧; 已被顺路扫描覆盖的站会跳过。
anchors = 1500*[cos((0:5)*pi/3);sin((0:5)*pi/3)];
a = (0:63)*2*pi/64;
domain = 1800/cos(pi/64)*[cos(a);sin(a)]; % 目标圆的外接多边形
reply = act('/enter'); limit = reply.remaining_real_duration_s;
reason = 'coverage_complete';
while true
    if toc(clock0)>limit-10, reason = 'real_time_limit'; break; end
    if sum(done)==16, reason = 'all_16_cleared'; break; end
    newlyCovered = vecnorm(unseen-pos)<=coverR;
    pending = find(known & ~done);
    % 新覆盖格子总面积至少 0.8 平方公里, 或无可追踪目标时停下扫描。
    % 扫描中顺便更新已知源的交会区域, 共享一次移动带来的观测机会。
    if any(newlyCovered) && (sum(newlyCovered)*h^2>=8e5 || isempty(pending))
        order = [channel,setdiff(1:20,channel,'stable')];
        for k = order
            if ~done(k) && (~known(k)||radii(k)>60), observe(pos,k); end
        end
        unseen(:,newlyCovered) = [];
        pending = find(known & ~done);
    end
    if isempty(pending) && isempty(unseen), break; end
    useful = any(hypot(unseen(1,:)-anchors(1,:), ...
        unseen(2,:)-anchors(2,:))<=coverR,1);
    % 把已知目标和必要补查站排在同一条开放路径中, 避免最后折返补查。
    % 最近邻给初始路线, 2-opt 只做路段反转; 每完成一项再更新这条路线。
    tasks = [centers(:,pending),anchors(:,useful)];
    tour = zeros(1,size(tasks,2)); left = 1:size(tasks,2); q = pos;
    for i = 1:numel(tour)
        [~,j] = min(vecnorm(tasks(:,left)-q));
        tour(i)=left(j); q=tasks(:,left(j)); left(j)=[];
    end
    improved = true;
    while improved
        improved = false;
        for i = 1:numel(tour)-1
            for j = i+1:numel(tour)
                route=[pos,tasks(:,tour)];
                old=norm(route(:,i)-route(:,i+1));
                new=norm(route(:,i)-route(:,j+1));

```

```

        if j<numel(tour)
            old=old+norm(route(:,j+1)-route(:,j+2));
            new=new+norm(route(:,i+1)-route(:,j+2));
        end
        if new<old-1e-6, tour(i:j)=fliplr(tour(i:j)); improved=true; end
    end
end
end
j=tour(1);
if j>numel(pending)
    q=tasks(:,j); order=[channel,setdiff(1:20,channel,'stable')];
    for k=order
        if ~done(k) && (~known(k)||radii(k)>60), observe(q,k); end
    end
    unseen(:,vecnorm(unseen-q)<=coverR)=[];
    continue;
end
k = pending(j);
for step = 1:12
    if done(k), break; end
    c = centers(:,k); r = radii(k);
    if r<=60
        % 小区域先尝试光学清除; 失败也确实计入 3 s, 不伪装成成功。
        if clearAt(c,k), break; end
        observe(pos,k);
    else
        % 向区域中心靠近, 并侧移最多 80 m 增加交会角。
        u = c-pos; if norm(u)<1e-9, u = [1;0]; end
        u = u/norm(u); q = c+min(80,r/2)*[-u(2);u(1)];
        observe(q,k);
    end
end
if ~done(k)
    % 极端几何退化兜底: 25 m 方格的最远覆盖距离为 17.68 m < 20 m。
    % 扫描定位多边形的整个包围矩形; 不用随机重测同一点来减小误差。
    V = regions{k}; lo = min(V,[],2); hi = max(V,[],2);
    for y = lo(2):25:hi(2)+25
        xs = lo(1):25:hi(1)+25;
        if abs(xs(end)-pos(1))<abs(xs(1)-pos(1)), xs = fliplr(xs); end
        for x = xs
            if clearAt([x;y],k), break; end
        end
        if done(k), break; end
    end
    assert(done(k),'光学覆盖兜底失败: 请检查测向误差或接口数据。');
end
end
end

```

```

act('/exit');
out = struct('cleared',sum(done),'totalTime',vt,'averageTime',vt/max(1,sum(done)), ...
    'complete',all(~known|done) && (isempty(unseen)||sum(done)==16), ...
    'reason',reason,'runtime',toc(clock0),'discoveryTime',discovered, ...
    'clearTime',cleared,'history',{history},'remainingCells',size(unseen,2));
if ~local
    stamp=char(datetime('now','Format','yyyyMMdd_HH:mm:ss'));
    save(fullfile(root,['problem3_run_' stamp '.mat']),'out');
    fprintf('清除 %d 个, 总时间 %.2f s, 平均 %.2f s, 完整排查=%d\n', ...
        out.cleared,out.totalTime,out.averageTime,out.complete);
end

function observe(q,k)
    % 只从接口响应取得信息; 无信号不推断距离为 1500 m 以外。
    z = act('/measure',q,k);
    if strcmp(z.measure_result,'no_signal'), return; end
    if ~known(k), known(k)=true; discovered(k)=vt; regions{k}=domain; end
    if strcmp(z.measure_result,'near')
        assert(clearAt(q,k),'near 后光学清除失败。'); return;
    end
    % 接口示向度保留两位小数: 将  $\pm 1^\circ$  加宽到  $\pm 1.005^\circ$  容纳舍入。
    th = deg2rad(z.svd_deg);
    [n1,b1,n2,b2] = wedge_planes(q,th,deg2rad(1.005));
    V = clip_halfplane(regions{k},n1,b1+1e-7);
    V = clip_halfplane(V,n2,b2+1e-7);
    u = [cos(th);sin(th)];
    V = clip_halfplane(V,u,u'*q+1500+1e-7);
    assert(~isempty(V),'交会区域为空: 观测与模型不相容。');
    regions{k} = V;
    D = diameter_many({V}); % 直接使用问题一的直径算法
    if D<=20
        c = V(:,1); r = D; % 任一顶点到区域内所有点均不超过 D
    else
        [c,r] = welzl_mec(V,cfg); % 直接使用问题二的包围圆工具
    end
    centers(:,k)=c; radii(k)=max(r,max(vecnorm(V-c)));
end

function success = clearAt(q,k)
    z = act('/clear',q,k);
    success = strcmp(z.clear_result,'success');
    if success, done(k)=true; cleared(k)=vt; end
end

function z = act(path,q,k)
    % 一个动作一个 ID; 通信失败重试同一请求, 避免重复计时或移动。
    if ~strcmp(path,'/exit') && toc(clock0)>limit-5

```

```

        error('现实时间不足，停止发送新动作。');
    end
    serial = serial+1;
    request = struct('arena_id','default','robot_id',robotId, ...
        'request_id',sprintf('p3-%d',serial));
    if nargin>1
        request.position = struct('x',q(1),'y',q(2)); request.channel=k;
    end
    for attempt = 1:3
        try
            if local, z=transport(path,request);
            else, z=webwrite([baseUrl path],request,opt); end
            break;
        catch err
            if attempt==3, rethrow(err); end
        end
    end
    assert(z.accepted,'模拟器拒绝请求: %s',jsonencode(z));
    vt = z.virtual_time_s;
    if nargin>1, pos=q; end
    if strcmp(path,'/measure'), channel=k; end % clear 不切换测向频道
    history(end+1,:) = {path,request,z};
end
end
end

```

```

function out = problem3_solver_optimized(robotId, baseUrl)
% 问题三优化版：短半径覆盖骨架 + 动态开放路径。
% 官方模拟器就绪后: out = problem3_solver_optimized('你们的参赛队号');
% 本地公平对比请运行 problem3_compare_monte_carlo。
% 基础款 problem3_solver.m 保持不变；本文件继续复用问题一、二的几何工具。
if nargin < 1, error('请传入参赛队号；本地测试请运行 problem3_compare_monte_carlo。'); end
if nargin < 2, baseUrl = 'http://127.0.0.1:2026'; end
root = fileparts(mfilename('fullpath'));
addpath(fullfile(root,'问题1_交会定位'));
addpath(genpath(fullfile(root,'问题2_第二检测点选择')));
cfg = config_problem2();
local = isa(robotId,'function_handle');
if local, transport = robotId; robotId = 'local'; end
opt = weboptions('MediaType','application/json','Timeout',5);
pos = [0;0]; channel = 1; vt = 0; serial = 0; limit = inf; clock0 = tic;
known = false(1,20); done = known; regions = cell(1,20);
centers = nan(2,20); radii = inf(1,20); discovered = nan(1,20);
cleared = nan(1,20); history = cell(0,3);

% 用20 m方格给连续圆域建立覆盖证书。格子外接半径为20/sqrt(2) m:
% 若格心距扫描点不超过1000-margin, 则整个格子都在保守接收圆内。

```

```

h = 20; margin = h/sqrt(2);
[gx,gy] = meshgrid(-1800:h:1800);
unseen = [gx(:)';gy(:)'];
unseen = unseen(:,vecnorm(unseen)<=1800+margin);
coverR = 1000-margin-1e-6;

% 原点和正六边形构成必达搜索骨架。首轮在发现的目标越少，说明
% 目标越可能偏外，环站就越靠外；半径限制在[1200,1500] m。
% 对最小半径1200 m，外层最坏格心到最近环站的距离上界为980.05 m，
% 小于coverR=985.86 m；更大的环站半径也通过相同不等式验证。
% 因而自适应选择仍保证发现接收半径至少1000 m的任意全向源。
anchors = zeros(2,0); anchorRadius = nan;
a = (0:63)*2*pi/64;
domain = 1800/cos(pi/64)*[cos(a);sin(a)]; % 目标圆的外接多边形
reply = act('/enter'); limit = reply.remaining_real_duration_s;
reason = 'coverage_complete';
while true
    if toc(clock0)>limit-10, reason = 'real_time_limit'; break; end
    if sum(done)==16, reason = 'all_16_cleared'; break; end
    newlyCovered = vecnorm(unseen-pos)<=coverR;
    pending = find(known & ~done);
    % 顺路覆盖新增面积较大时扫描；没有已知目标时立即扫描以继续发现目标。
    if any(newlyCovered) && (sum(newlyCovered)*h^2>=8e5 || isempty(pending))
        order = [channel,setdiff(1:20,channel,'stable')];
        for k = order
            if ~done(k) && (~known(k)||radii(k)>60), observe(pos,k); end
        end
        unseen(:,newlyCovered) = [];
        pending = find(known & ~done);
    end
    if isempty(anchors)
        anchorRadius=max(1200,1500-60*sum(known));
        anchors=anchorRadius*[cos((0:5)*pi/3);sin((0:5)*pi/3)];
    end
    if isempty(pending) && isempty(unseen), break; end
    useful = any(hypot(unseen(1,:)-anchors(1,:), ...
        unseen(2,:)-anchors(2,:))<=coverR,1);

    % 把待清除目标与尚有覆盖贡献的骨架站放入同一条开放路径。
    tasks = [centers(:,pending),anchors(:,useful)];
    tour = zeros(1,size(tasks,2)); left = 1:size(tasks,2); q = pos;
    for i = 1:numel(tour)
        [~,j] = min(vecnorm(tasks(:,left)-q));
        tour(i)=left(j); q=tasks(:,left(j)); left(j)=[];
    end
    improved = true;
    while improved

```

```

improved = false;
for i = 1:numel(tour)-1
    for j = i+1:numel(tour)
        route=[pos,tasks(:,tour)];
        old=norm(route(:,i)-route(:,i+1));
        new=norm(route(:,i)-route(:,j+1));
        if j<numel(tour)
            old=old+norm(route(:,j+1)-route(:,j+2));
            new=new+norm(route(:,i+1)-route(:,j+2));
        end
        if new<old-1e-6, tour(i:j)=fliplr(tour(i:j)); improved=true; end
    end
end
j=tour(1);
if j>numel(pending)
    q=tasks(:,j); order=[channel,setdiff(1:20,channel,'stable')];
    for k=order
        if ~done(k) && (~known(k)||radii(k)>60), observe(q,k); end
    end
    unseen(:,vecnorm(unseen-q)<=coverR)=[];
    continue;
end

k = pending(j);
for step = 1:12
    if done(k), break; end
    c = centers(:,k); r = radii(k);
    if r<=60
        if clearAt(c,k), break; end
        observe(pos,k);
    else
        % 沿用基础款的局部定位动作，只检验搜索骨架收缩的净收益。
        u=c-pos; if norm(u)<1e-9, u=[1;0]; end
        u=u/norm(u); observe(c+min(80,r/2)*[-u(2);u(1)],k);
    end
end
if ~done(k)
    % 数值退化兜底：25 m光学方格最远覆盖距离17.68 m，小于20 m。
    V=regions{k}; lo=min(V,[],2); hi=max(V,[],2);
    for y=lo(2):25:hi(2)+25
        xs=lo(1):25:hi(1)+25;
        if abs(xs(end)-pos(1))<abs(xs(1)-pos(1)), xs=fliplr(xs); end
        for x=xs
            if clearAt([x;y],k), break; end
        end
        if done(k), break; end
    end
end

```

```

        end
        assert(done(k),'光学覆盖兜底失败：请检测测向误差或接口数据。');
    end
end
act('/exit');
out = struct('cleared',sum(done),'totalTime',vt,'averageTime',vt/max(1,sum(done)), ...
    'complete',all(~known|done) && (isempty(unseen)||sum(done)==16), ...
    'reason',reason,'runtime',toc(clock0),'discoveryTime',discovered, ...
    'clearTime',cleared,'history',{history},'remainingCells',size(unseen,2), ...
    'anchorRadius',anchorRadius);
if ~local
    stamp=char(datetime('now','Format','yyyyMMdd_HH:mm:ss'));
    save(fullfile(root,['problem3_optimized_run_' stamp '.mat']),'out');
    fprintf('优化版：清除%d个，总时间%.2f s，平均%.2f s，完整排查=%d\n', ...
        out.cleared,out.totalTime,out.averageTime,out.complete);
end

function observe(q,k)
    z=act('/measure',q,k);
    if strcmp(z.measure_result,'no_signal'), return; end
    if ~known(k), known(k)=true; discovered(k)=vt; regions{k}=domain; end
    if strcmp(z.measure_result,'near')
        assert(clearAt(q,k),'near 后光学清除失败。'); return;
    end
    th=deg2rad(z.svd_deg);
    [n1,b1,n2,b2]=wedge_planes(q,th,deg2rad(1.005));
    V=clip_halfplane(regions{k},n1,b1+1e-7);
    V=clip_halfplane(V,n2,b2+1e-7);
    u=[cos(th);sin(th)];
    V=clip_halfplane(V,u,u'*q+1500+1e-7);
    assert(~isempty(V),'交会区域为空：观测与模型不相容。');
    regions{k}=V; D=diameter_many({V});
    if D<=20, c=V(:,1); r=D;
    else, [c,r]=welzl_mec(V,cfg); end
    centers(:,k)=c; radii(k)=max(r,max(vecnorm(V-c)));
end

function success=clearAt(q,k)
    z=act('/clear',q,k); success=strcmp(z.clear_result,'success');
    if success, done(k)=true; cleared(k)=vt; end
end

function z=act(path,q,k)
    if ~strcmp(path,'/exit') && toc(clock0)>limit-5
        error('现实时间不足，停止发送新动作。');
    end
    serial=serial+1;

```

```

request=struct('arena_id','default','robot_id',robotId, ...
    'request_id',sprintf('p3opt-%d',serial));
if nargin>1
    request.position=struct('x',q(1),'y',q(2)); request.channel=k;
end
for attempt=1:3
    try
        if local, z=transport(path,request);
        else, z=webwrite([baseUrl path],request,opt); end
        break;
    catch err
        if attempt==3, rethrow(err); end
    end
end
assert(z.accepted,'模拟器拒绝请求: %s',jsonencode(z));
vt=z.virtual_time_s;
if nargin>1, pos=q; end
if strcmp(path,'/measure'), channel=k; end
history(end+1,:)={path,request,z};
end
end

```

2.4 问题四相关代码

```

function [comparison,summary] = problem4_compare_split_monte_carlo(trials,seed,scenario)
% 同一批问题四案例上配对比较基础款与“全向优先/定向分流”优化款。
% [comparison,summary]=problem4_compare_split_monte_carlo(100,20260912,'uniform');
% scenario: uniform / edge / all_directional / extreme。
if nargin<1, trials=100; end
if nargin<2, seed=20260912; end
if nargin<3, scenario='uniform'; end
validateattributes(trials',{'numeric'},{'scalar','integer','positive'});
assert(ismember(scenario',{'uniform','edge','all_directional','extreme'}),'未知场景。');
root=fileparts(mfilename('fullpath')); addpath(root);
world=RandStream('mt19937ar','Seed',seed);
oldRng=rng; cleanup=onCleanup(@() rng(oldRng));
values=zeros(2*trials,21); algorithms=strings(2*trials,1); errors=strings(2*trials,1);
solvers={@problem4_solver,@problem4_solver_split}; labels=["base" "split"];
for trial=1:trials
    n=randi(world,[10,16]); channels=randperm(world,20,n);
    theta=2*pi*rand(world,1,n); rho=1800*sqrt(rand(world,1,n));
    xy=[rho.*cos(theta);rho.*sin(theta)]; radius=1000+500*rand(world,1,n);
    nDir=randi(world,[1,n-1]); directed=false(1,n);
    directed(randperm(world,n,nDir))=true; emit=2*pi*rand(world,1,n);
    if strcmp(scenario,'edge')

```

```

        xy=1800*[cos(theta);sin(theta)]; radius(:)=1000; emit(directed)=theta(directed);
    elseif strcmp(scenario,'all_directional')
        directed(:)=true;
    elseif strcmp(scenario,'extreme')
        radius(:)=1000; rho=1800*rand(world,1,n).^(1/4);
        xy=[rho.*cos(theta);rho.*sin(theta)]; emit(directed)=theta(directed);
    end
    for a=1:2
        rng(seed+trial);
        [m,err]=runCase(solvers{a},channels,xy,radius,directed,emit, ...
            seed+104729*trial,scenario);
        row=2*(trial-1)+a; values(row,:)= [trial,n,sum(directed),m];
        algorithms(row)=labels(a); errors(row)=err;
    end
    b=values(2*trial-1,6); s=values(2*trial,6);
    fprintf('%4d/%d | 定向%2d/%2d | 基础 %.1f | 分流 %.1f | 节省 %.1f s/源\n', ...
        trial,trials,sum(directed),n,b,s,b-s);
end
names={'Trial','N','Directional','Cleared','Total_s','Average_s','Runtime_s', ...
    'Complete','Move_s','Switch_s','Measure_s','OpticalLaser_s','StationsVisited', ...
    'Fallbacks','RaySweeps','DirectionObservations','GlobalMove_s','LocalMove_s', ...
    'DirectionalMode','CertifiedDirectional','P2Tests'};
comparison=array2table(values,'VariableNames',names);
comparison.Algorithm=algorithms; comparison.Error=errors;
comparison=movevars(comparison',{'Algorithm','Error'},'After','Trial');
b=comparison(comparison.Algorithm=="base",:); s=comparison(comparison.Algorithm=="split",:);
summary=table(labels',[mean(b.Average_s);mean(s.Average_s)], ...
    [sum(b.Total_s)/sum(b.Cleared);sum(s.Total_s)/sum(s.Cleared)], ...
    [mean(b.Total_s);mean(s.Total_s)], [mean(b.Move_s);mean(s.Move_s)], ...
    [mean(b.Measure_s+b.Switch_s);mean(s.Measure_s+s.Switch_s)], ...
    [mean(b.GlobalMove_s);mean(s.GlobalMove_s)], [mean(b.LocalMove_s);mean(s.LocalMove_s)], ...
    [mean(b.Complete);mean(s.Complete)], [mean(b.Average_s<300);mean(s.Average_s<300)], ...
    'VariableNames',{'Algorithm','MeanAverage_s','PooledAverage_s','MeanTotal_s', ...
    'MeanMove_s','MeanMeasureSwitch_s','MeanGlobalMove_s','MeanLocalMove_s', ...
    'FullClearRate','Below300Rate'});
delta=b.Average_s-s.Average_s;
fprintf('\n%s: 基础 %.2f, 分流 %.2f s/源; 平均节省 %.2f s/源 (%.1f%%) 。\n', ...
    scenario,mean(b.Average_s),mean(s.Average_s),mean(delta),100*mean(delta)/mean(b.Average_s));
fprintf('全清率: 基础 %.1f%%, 分流 %.1f%%; 分流胜出 %.1f%% 的配对案例.\n', ...
    100*mean(b.Complete),100*mean(s.Complete),100*mean(delta>0));
folder=fullfile(root,'问题4优化结果'); if ~isfolder(folder), mkdir(folder); end
prefix=fullfile(folder,['problem4_split_compare_' scenario]);
writetable(comparison,[prefix '_runs.csv']); writetable(summary,[prefix '_summary.csv']);
save([prefix '.mat'],'comparison','summary','seed','scenario');
f=figure('Color','w','Visible','off','Position',[100 100 900 650]); tiledlayout(2,1);
nexttile; plot(b.Average_s,'o-','DisplayName','基础款'); hold on;
plot(s.Average_s,'o-','DisplayName','分流款'); yline(300,'r--','300 s');

```

```

ylabel('Time per source (s)'); legend('Location','best'); title(['Problem 4: ' scenario]);
    grid on;
nexttile; bar(delta); yline(0,'k-'); xlabel('Paired trial');
ylabel('Base - split (s/source)'); grid on;
set(findall(f,'Type','axes'),'Color','w','XColor','k','YColor','k','GridColor',[.3 .3 .3]);
set(findall(f,'Type','text'),'Color','k'); exportgraphics(f,[prefix '.png'],'Resolution',150);
    close(f);
end

function [metrics,errText]=runCase(solver,channels,xy,radius,directed,emit,noiseSeed,scenario)
% 真值仅保留在封闭模拟器中；两算法接收完全相同的接口反馈规则。
n=numel(channels); live=true(1,n); p=[0;0]; ch=1; t=0; parts=zeros(1,4);
errText=""; result=struct(); start=tic;
try
    result=solver(@simulator);
catch err
    errText=string(getReport(err,'extended','hyperlinks','off'));
end
count=sum(~live); average=t/max(1,count); if count==0, average=inf; end
complete=~any(live)&&isfield(result,'complete')&&result.complete;
v=nan(1,9);
if isfield(result,'stationsVisited'), v(1)=result.stationsVisited; end
if isfield(result,'fallbackCount'), v(2)=result.fallbackCount; end
if isfield(result,'raySweepCount'), v(3)=result.raySweepCount; end
if isfield(result,'observations'), v(4)=sum(result.observations); end
if isfield(result,'globalDistance'), v(5)=result.globalDistance/5; end
if isfield(result,'localDistance'), v(6)=result.localDistance/5; end
if isfield(result,'directionalModeCount'), v(7)=result.directionalModeCount; end
if isfield(result,'certifiedDirectionalCount'), v(8)=result.certifiedDirectionalCount; end
if isfield(result,'p2Tests'), v(9)=result.p2Tests; end
metrics=[count,t,average,toc(start),complete,parts,v];
assert(abs(t-sum(parts))<1e-6,'模拟计时分项不守恒。');

function z=simulator(path,request)
    z=struct('accepted',true,'virtual_time_s',t);
    if strcmp(path,'/enter'), z.remaining_real_duration_s=1200; return; end
    if strcmp(path,'/exit'), z.exit_reason='user_exit'; return; end
    q=[request.position.x;request.position.y]; k=request.channel;
    dt=norm(q-p)/5; parts(1)=parts(1)+dt; t=t+dt; p=q;
    j=find(channels==k&live,1); d=inf; covered=false;
    if ~isempty(j)
        d=norm(xy(:,j)-p); e=[cos(emit(j));sin(emit(j))];
        covered=~directed(j)|| (p-xy(:,j))'*e>=-1e-9;
    end
    if strcmp(path,'/measure')
        sw=double(ch~=k); ch=k; t=t+sw+5;
        parts(2)=parts(2)+sw; parts(3)=parts(3)+5;
    end
end

```

```

        if isempty(j)||d>radius(j)||~covered
            z.measure_result='no_signal';
        elseif d<=5
            z.measure_result='near';
        else
            w=sin(q(1)*12.9898+q(2)*78.233+k*37.719+noiseSeed*.013)*43758.5453;
            err=2*(w-floor(w))-1;
            if strcmp(scenario,'extreme'), err=sign(err); end
            u=xy(:,j)-p; z.measure_result='direction';
            z.svd_deg=mod(round(atan2d(u(2),u(1))+err,2),360);
        end
    else
        ok=~isempty(j)&&d<=20; dt=3+2*ok; t=t+dt; parts(4)=parts(4)+dt;
        if ok, live(j)=false; z.clear_result='success';
        else, z.clear_result='no_target_in_range'; end
    end
    z.virtual_time_s=t;
end
end

```

```

function [runs,sources] = problem4_monte_carlo(trials,seed,scenario)
% 问题四蒙特卡洛：只通过模拟接口调用 problem4_solver，不向策略泄露真值。
% [runs,sources] = problem4_monte_carlo(100,20260912,'uniform');
% 场景：uniform（混合随机）、edge（边界外向压力）、
%       all_directional（全定向压力）、extreme（最小半径和端点误差）。
% 主要输出每局总时长、每源平均时长、首次发现及清除时刻。
if nargin<1, trials=100; end
if nargin<2, seed=20260912; end
if nargin<3, scenario='uniform'; end
validateattributes(trials',{'numeric'},{'scalar','integer','positive'});
assert(ismember(scenario,{'uniform','edge','all_directional','extreme'}),'未知场景。');
root=fileparts(mfilename('fullpath')); addpath(root);
world=RandStream('mt19937ar','Seed',seed);
oldRng=rng; cleanup=onCleanup(@() rng(oldRng)); rng(seed);
records=zeros(trials,18); sourceRows=zeros(0,7); errors=strings(trials,1);
for trial=1:trials
    n=randi(world,[10,16]); channels=randperm(world,20,n);
    theta=2*pi*rand(world,1,n); rho=1800*sqrt(rand(world,1,n));
    xy=[rho.*cos(theta);rho.*sin(theta)]; radius=1000+500*rand(world,1,n);
    nDir=randi(world,[1,n-1]); directed=false(1,n);
    directed(randperm(world,n,nDir))=true; emit=2*pi*rand(world,1,n);
    if strcmp(scenario,'edge')
        xy=1800*[cos(theta);sin(theta)]; radius(:)=1000;
        emit(directed)=theta(directed); % 定向波束朝圆外，专门检验最难边界
    elseif strcmp(scenario,'all_directional')
        directed(:)=true;
    end
end

```

```

elseif strcmp(scenario,'extreme')
    radius(:)=1000; rho=1800*rand(world,1,n).^(1/4);
    xy=[rho.*cos(theta);rho.*sin(theta)];
    emit(directed)=theta(directed);
end
isDir=false(1,20); isDir(channels)=directed;
live=true(1,n); p=[0;0]; ch=1; t=0; parts=zeros(1,4);
noise=containers.Map('KeyType','char','ValueType','double');
noiseRng=RandStream('mt19937ar','Seed',seed+trial); result=struct(); why="";
start=tic; complete=false; tail=nan;
try
    result=problem4_solver(@simulator);
    complete=result.complete&&~any(live);
    ids=find(isfinite(result.clearTime));
    [times,order]=sort(result.clearTime(ids)); ids=ids(order);
    gaps=diff([0,times]); if ~isempty(times), tail=t-times(end); end
    sourceRows=[sourceRows; repmat(trial,numel(ids),1),ids', ...
        isDir(ids)',result.discoveryTime(ids)',times',gaps', ...
        (times-result.discoveryTime(ids))']]; %#ok<AGROW>
catch err
    why=string(err.message); % 失败局保留在统计中，不静默删除
end
count=sum(~live); avg=t/max(1,count); if count==0, avg=inf; end
stationVisited=nan; fallback=nan; raySweep=nan; observations=nan;
globalMove=nan; localMove=nan;
if isfield(result,'stationsVisited'), stationVisited=result.stationsVisited; end
if isfield(result,'fallbackCount'), fallback=result.fallbackCount; end
if isfield(result,'raySweepCount'), raySweep=result.raySweepCount; end
if isfield(result,'observations'), observations=sum(result.observations); end
if isfield(result,'globalDistance'), globalMove=result.globalDistance/5; end
if isfield(result,'localDistance'), localMove=result.localDistance/5; end
records(trial,:)= [n,sum(directed),count,t,avg,toc(start),complete,parts,tail, ...
    stationVisited,fallback,raySweep,observations,globalMove,localMove]; errors(trial)=why;
fprintf('%4d/%2d | 定向%2d/%2d | 清除%2d | 总%.1f s | 平均%.1f s | 全清%d\n', ...
    trial,trials,sum(directed),n,count,t,avg,complete);
assert(abs(t-sum(parts))<1e-6,'模拟计时分项不守恒。');
end
runs=array2table(records,'VariableNames',{'N','Directional','Cleared','Total_s', ...
    'Average_s','Runtime_s','Complete','Move_s','Switch_s','Measure_s', ...
    'OpticalLaser_s','FinalCheck_s','StationsVisited','Fallbacks','RaySweeps', ...
    'DirectionObservations','GlobalMove_s','LocalMove_s'});
runs.Error=errors;
sources=array2table(sourceRows,'VariableNames',{'Trial','Channel','IsDirectional', ...
    'Discovery_s','Clear_s','ClearInterval_s','DiscoveryToClear_s'});
folder=fullfile(root,'问题4结果'); if ~isfolder(folder), mkdir(folder); end
prefix=fullfile(folder,['problem4_mc_' scenario]);
writetable(runs,[prefix '_runs.csv']); writetable(sources,[prefix '_sources.csv']);

```

```

save([prefix '.mat'], 'runs', 'sources', 'seed', 'scenario', 'result');
fprintf('\n全清率 %.1f%%; 逐局平均 %.2f s/源; 汇总平均 %.2f s/源.\n', ...
    100*mean(runs.Complete), mean(runs.Average_s), sum(runs.Total_s)/sum(runs.Cleared));
fprintf('全清且平均<300 s: %.1f%%; 最慢一局 %.2f s/源; 平均兜底 %.2f 次.\n', ...
    100*mean(runs.Complete & runs.Average_s<300), max(runs.Average_s), mean(runs.Fallbacks));
fprintf('平均耗时分解: 全局移动 %.1f, 局部移动 %.1f, 测量及切换 %.1f, 光学清除 %.1f s.\n', ...
    mean(runs.GlobalMove_s), mean(runs.LocalMove_s), ...
    mean(runs.Measure_s+runs.Switch_s), mean(runs.OpticalLaser_s));
f=figure('Color', 'w', 'Visible', 'off', 'Position', [100 100 850 620]);
tiledlayout(2,1);
nexttile; plot(runs.Total_s, 'o-'); ylabel('Total time (s)'); grid on;
title(sprintf('Problem 4 Monte Carlo: %s', scenario));
nexttile; plot(runs.Average_s, 'o-'); yline(300, 'r--', '300 s');
xlabel('Trial'); ylabel('Time per source (s)'); grid on;
set(findall(f, 'Type', 'axes'), 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'GridColor', [.3 .3 .3]);
set(findall(f, 'Type', 'text'), 'Color', 'k');
exportgraphics(f, [prefix '.png'], 'Resolution', 150); close(f);

function z=simulator(path,request)
    % 真值封闭在接口中; 定向源仅在接收半径内且检测点位于发射半平面时可见。
    z=struct('accepted', true, 'virtual_time_s', t);
    if strcmp(path, '/enter'), z.remaining_real_duration_s=1200; return; end
    if strcmp(path, '/exit'), z.exit_reason='user_exit'; return; end
    q=[request.position.x; request.position.y]; k=request.channel;
    dt=norm(q-p)/5; parts(1)=parts(1)+dt; t=t+dt; p=q;
    j=find(channels==k&live, 1); d=inf; covered=false;
    if ~isempty(j)
        d=norm(xy(:,j)-p); e=[cos(emit(j)); sin(emit(j))];
        covered=~directed(j) || (p-xy(:,j))*e>=-1e-9;
    end
    if strcmp(path, '/measure')
        sw=double(ch~=k); ch=k; t=t+sw+5;
        parts(2)=parts(2)+sw; parts(3)=parts(3)+5;
        if isempty(j) || d>radius(j) || ~covered
            z.measure_result='no_signal';
        elseif d<=5
            z.measure_result='near';
        else
            key=sprintf('%d: %.17g: %.17g', k, p(1), p(2));
            if ~isKey(noise, key)
                e=2*rand(noiseRng)-1;
                if strcmp(scenario, 'extreme'), e=sign(e); end
                noise(key)=e;
            end
            v=xy(:,j)-p; z.measure_result='direction';
            z.svd_deg=mod(round(atan2d(v(2), v(1))+noise(key), 2), 360);
        end
    end
end

```

```

else
    ok=~isempty(j)&&d<=20; dt=3+2*ok; t=t+dt; parts(4)=parts(4)+dt;
    if ok, live(j)=false; z.clear_result='success';
    else, z.clear_result='no_target_in_range'; end
end
z.virtual_time_s=t;
end
end

function out = problem4_solver(robotId, baseUrl)
% 问题四基础款：双环稳健搜索 + 成对侧向交会，兼容全向和180度定向源。
% 官方模拟器：out = problem4_solver('你们的参赛队号');
% 本地检验：problem4_monte_carlo(100,20260912,'uniform');
% 直接复用问题一的 diameter_many 和问题二的半平面、最小包围圆算法。
if nargin<1, error('请传入参赛队号；本地测试请运行 problem4_monte_carlo。'); end
if nargin<2, baseUrl='http://127.0.0.1:2026'; end
root=fileparts(mfilename('fullpath'));
addpath(fullfile(root,'问题1_交会定位'));
addpath(genpath(fullfile(root,'问题2_第二检测点选择')));
cfg=config_problem2();
local=isa(robotId,'function_handle');
if local, transport=robotId; robotId='local'; end
opt=weboptions('MediaType','application/json','Timeout',5);
pos=[0;0]; channel=1; vt=0; serial=0; limit=inf; clock0=tic;
inLocal=false; globalDistance=0; localDistance=0;
known=false(1,20); done=known; regions=cell(1,20);
centers=nan(2,20); radii=inf(1,20); obsCount=zeros(1,20);
lastPos=nan(2,20); lastTheta=nan(1,20);
firstPos=nan(2,20); firstTheta=nan(1,20);
discovered=nan(1,20); cleared=nan(1,20); history=cell(0,3);
fallbackCount=0; raySweepCount=0;

% 搜索覆盖证书：外环正十二边形内切圆半径
%  $1865 \times \cos(15^\circ) = 1801.46 > 1800$ ；内外环错开15度。
% 原点、内环和外环的三角剖分最大边长约979 m，小于最小接收半径1000 m。
% 任意源位于某个小三角形内；任意过该源的180度半平面至少含一个顶点，
% 所以不论定向源朝向如何，至少有一个距离不超过1000 m的搜索站能收到信号。
a=(0:11)*pi/6;
stations=[0;0],950*[cos(a);sin(a)],1865*[cos(a+pi/12);sin(a+pi/12)];
visited=false(1,size(stations,2));
ang=(0:71)*2*pi/72;
domain=1800/cos(pi/72)*[cos(ang);sin(ang)]; % 圆域的保守外接多边形

reply=act('/enter'); limit=reply.remaining_real_duration_s;
reason='search_complete';
while true

```

```

if toc(clock0)>limit-10, reason='real_time_limit'; break; end
if sum(done)==16, reason='all_16_cleared'; break; end
left=find(~visited);
pending=find(known & ~done);
if isempty(left)&&isempty(pending), break; end

% 把待清除源与剩余搜索站放入同一条开放路线；目标只有在顺路成为
% 下一节点时才追踪，后续搜索站也可在此前免费补充新的方位线。
tasks=[centers(:,pending),stations(:,left)];
tour=zeros(1,size(tasks,2)); pool=1:size(tasks,2); q=pos;
for routeStep=1:numel(tour)
    [~,pick]=min(vecnorm(tasks(:,pool)-q));
    tour(routeStep)=pool(pick); q=tasks(:,pool(pick)); pool(pick)=[];
end
improved=true;
while improved
    improved=false; route=[pos,tasks(:,tour)];
    for swapStart=1:numel(tour)-1
        for swapEnd=swapStart+1:numel(tour)
            old=norm(route(:,swapStart)-route(:,swapStart+1));
            new=norm(route(:,swapStart)-route(:,swapEnd+1));
            if swapEnd<numel(tour)
                old=old+norm(route(:,swapEnd+1)-route(:,swapEnd+2));
                new=new+norm(route(:,swapStart+1)-route(:,swapEnd+2));
            end
            if new<old-1e-7
                tour(swapStart:swapEnd)=fliplr(tour(swapStart:swapEnd)); improved=true;
                route=[pos,tasks(:,tour)];
            end
        end
    end
end
next=tour(1);
if next<=numel(pending)
    k=pending(next); inLocal=true; localize(k); inLocal=false;
    continue;
end
stationId=left(next-numel(pending));
q=stations(:,stationId); visited(stationId)=true;

% 未发现频道继续全查；已发现频道最多利用搜索站补到六条方位线，
% 此后等待路线中的追踪节点，避免重复测量同一频道。
list=find(~done & (~known | obsCount<6));
order=[channel,setdiff(list,channel,'stable')];
if ~ismember(channel,list), order=list; end
for k=order
    observe(q,k);
end

```

```

        if sum(done)==16, break; end
    end

end

if toc(clock0)<=limit-5, act('/exit'); end
out=struct('cleared',sum(done),'totalTime',vt,'averageTime',vt/max(1,sum(done)), ...
    'complete',(all(visited)||sum(done)==16) && all(~known|done), ...
    'reason',reason,'runtime',toc(clock0),'discoveryTime',discovered, ...
    'clearTime',cleared,'observations',obsCount,'finalRadius',radii,'history',{history}, ...
    'stationsVisited',sum(visited),'stationCount',size(stations,2), ...
    'fallbackCount',fallbackCount,'raySweepCount',raySweepCount, ...
    'globalDistance',globalDistance,'localDistance',localDistance);
if ~local
    stamp=char(datetime('now','Format','yyyyMMdd_HH:mm:ss'));
    save(fullfile(root,['problem4_run_' stamp '.mat']),'out');
    fprintf('问题四基础款: 清除%d个, 总时间%.2f s, 平均%.2f s, 完整排查=%d\n', ...
        out.cleared,out.totalTime,out.averageTime,out.complete);
end

function localize(k)
    % 沿最近测向线向前推进。两探测点关于测向线对称, 侧移量超过
    % 490*tan(1.005度), 所以真实射线穿过两点之间的线段; 只要源仍在
    % 前方, 180度发射半平面至少包含其中一点, 且两点均在接收半径内。
    for round=1:5
        if done(k), return; end
        if radii(k)<=20 && clearAt(centers(:,k),k), return; end
        q0=lastPos(:,k); th=lastTheta(k);
        if any(~isfinite(q0))||~isfinite(th), break; end
        U=min(1500,max(vecnorm(regions{k}-q0)));
        if U<=600, raySweep(k,q0,th,U); return; end
        u=[cos(th);sin(th)]; v=[-u(2);u(1)];
        advance=min(490,0.6*U); side=advance*tan(deg2rad(1.005))+3;
        probes=[q0+advance*u+side*v,q0+advance*u-side*v];
        if norm(probes(:,2)-pos)<norm(probes(:,1)-pos), probes=fliplr(probes); end
        hit=observe(probes(:,1),k);
        if ~done(k)&&~hit, hit=observe(probes(:,2),k); end
        if done(k), return; end
        if ~hit
            % 成对无信号时, 源必在推进截面之前; 无信号在这里作为
            % “一对互补实验”的联合结论使用, 而非单点排除。
            raySweep(k,q0,th,advance/cos(deg2rad(1.005))+1); return;
        end
        if clearAt(lastPos(:,k),k), return; end
    end
    if done(k), return; end

    % 兜底只使用第一次有效测向: 真源在长1500 m、半宽26.32 m的窄带内。

```

```

% 28 m方格的覆盖半径为19.80 m, 小于20 m清除半径, 因此必能清除。
fallbackCount=fallbackCount+1; u=[cos(firstTheta(k));sin(firstTheta(k))];
v=[-u(2);u(1)]; xs=0:28:1512; ys=[-42,-14,14,42];
% 四种蛇形起点中选离当前位置最近者, 避免无意义折返。
paths=cell(1,4); d0=inf(1,4); c=0;
for flipY=0:1
    yy=ys; if flipY, yy=fliplr(yy); end
    for flipX=0:1
        c=c+1; P=zeros(2,numel(xs)*numel(yy)); t=0;
        for r=1:numel(yy)
            xx=xs; if mod(r+flipX,2)==0, xx=fliplr(xx); end
            Q=firstPos(:,k)+u*xx+v*(yy(r)*ones(size(xx)));
            P(:,t+(1:numel(xx)))=Q; t=t+numel(xx);
        end
        paths{c}=P; d0(c)=norm(P(:,1)-pos);
    end
end
[~,c]=min(d0); P=paths{c};
for fallbackPoint=1:size(P,2)
    if clearAt(P(:,fallbackPoint),k), return; end
end
error('光学窄带兜底失败: 请检查测向误差或模拟器规则。');
end

function raySweep(k,q0,th,U)
% 已知源在长度U的单次测向误差带内。当U<=600 m时横向误差小于
% 10.53 m; 按清除圆余下的弦长设置纵向间距, 可用一条直线完全覆盖。
raySweepCount=raySweepCount+1; epsA=deg2rad(1.005);
halfWidth=U*sin(epsA); assert(halfWidth<20,'测向带过宽, 不能单线覆盖。');
step=1.98*sqrt(20^2-halfWidth^2);
xs=step/2:step:U+step/2;
u=[cos(th);sin(th)]; P=q0+u*xs;
if norm(P(:,end)-pos)<norm(P(:,1)-pos), P=fliplr(P); end
for sweepPoint=1:size(P,2)
    if clearAt(P(:,sweepPoint),k), return; end
end
error('测向带直线覆盖失败: 请检查误差上界或距离上界。');
end

function hit=observe(q,k)
% no_signal 可能由距离、朝向或空频道造成, 故不据此裁剪位置区域。
z=act('/measure',q,k); hit=~strcmp(z.measure_result,'no_signal');
if ~hit, return; end
if ~known(k)
    known(k)=true; discovered(k)=vt; regions{k}=domain;
end
if strcmp(z.measure_result,'near')

```

```

        assert(clearAt(q,k), 'near 后光学清除失败。'); return;
    end
    th=deg2rad(z.svd_deg); obsCount(k)=obsCount(k)+1;
    lastPos(:,k)=q; lastTheta(k)=th;
    if obsCount(k)==1, firstPos(:,k)=q; firstTheta(k)=th; end
    [n1,b1,n2,b2]=wedge_planes(q,th,deg2rad(1.005));
    V=clip_halfplane(regions{k},n1,b1+1e-7);
    V=clip_halfplane(V,n2,b2+1e-7);
    u=[cos(th);sin(th)]; V=clip_halfplane(V,u,u'*q+1500+1e-7);
    assert(~isempty(V), '交会区域为空：观测与误差模型不相容。');
    regions{k}=V; D=diameter_many({V});
    if D<=20, c=V(:,1); r=D;
    else, [c,r]=welzl_mec(V,cfg); end
    centers(:,k)=c; radii(k)=max(r,max(vecnorm(V-c)));
end

function success=clearAt(q,k)
    z=act('/clear',q,k); success=strcmp(z.clear_result,'success');
    if success, done(k)=true; cleared(k)=vt; end
end

function z=act(path,q,k)
    % 新动作使用新ID；网络故障只重试完全相同的请求和ID。
    if ~strcmp(path,'/exit')&&toc(clock0)>limit-5
        error('现实时间不足，停止发送新动作。');
    end
    serial=serial+1;
    request=struct('arena_id','default','robot_id',robotId, ...
        'request_id',sprintf('p4-%d',serial));
    if nargin>1
        request.position=struct('x',q(1),'y',q(2)); request.channel=k;
    end
    for attempt=1:3
        try
            if local, z=transport(path,request);
            else, z=webwrite([baseUrl path],request,opt); end
            break;
        catch err
            if attempt==3, rethrow(err); end
        end
    end
    assert(z.accepted, '模拟器拒绝请求： %s',jsonencode(z)); vt=z.virtual_time_s;
    if nargin>1
        if inLocal, localDistance=localDistance+norm(q-pos);
        else, globalDistance=globalDistance+norm(q-pos); end
        pos=q;
    end
end

```

```

        if strcmp(path, '/measure'), channel=k; end
        history(end+1,:)={path,request,z};
    end
end

```

```

function out = problem4_solver_split(robotId, baseUrl)
% 问题四分流优化版：信号稳定时沿用问题三全向定位，失联后转定向追踪。
% 官方模拟器：out = problem4_solver_split('你们的参赛队号');
% 配对检验：problem4_compare_split_monte_carlo(100,20260912,'uniform');
% 与基础款保持同一主结构；标有“分流改动”的部分是本版新增逻辑。
if nargin<1, error('请传入参赛队号；本地测试请运行配对蒙特卡洛程序。'); end
if nargin<2, baseUrl='http://127.0.0.1:2026'; end
root=fileparts(mfilename('fullpath'));
addpath(fullfile(root,'问题1_交会定位'));
addpath(genpath(fullfile(root,'问题2_第二检测点选择')));
cfg=config_problem2();
local=isa(robotId,'function_handle');
if local, transport=robotId; robotId='local'; end
opt=weboptions('MediaType','application/json','Timeout',5);
pos=[0;0]; channel=1; vt=0; serial=0; limit=inf; clock0=tic;
inLocal=false; globalDistance=0; localDistance=0;
known=false(1,20); done=known; regions=cell(1,20);
centers=nan(2,20); radii=inf(1,20); obsCount=zeros(1,20);
lastPos=nan(2,20); lastTheta=nan(1,20);
firstPos=nan(2,20); firstTheta=nan(1,20);
discovered=nan(1,20); cleared=nan(1,20); history=cell(0,3);
fallbackCount=0; raySweepCount=0;

% 分流改动：mode=1 表示仍按全向源处理，mode=2 表示转入定向稳健追踪。
mode=zeros(1,20); certifiedDirectional=false(1,20); omniRefineCount=0;

% 与基础款完全相同的25点确定性覆盖证书，不淘汰任何未发现频道。
a=(0:11)*pi/6;
stations=[[0;0],950*[cos(a);sin(a)],1865*[cos(a+pi/12);sin(a+pi/12)]];
visited=false(1,size(stations,2));
ang=(0:71)*2*pi/72;
domain=1800/cos(pi/72)*[cos(ang);sin(ang)];

reply=act('/enter'); limit=reply.remaining_real_duration_s;
reason='search_complete';
while true
    if toc(clock0)>limit-10, reason='real_time_limit'; break; end
    if sum(done)==16, reason='all_16_cleared'; break; end
    left=find(~visited); pending=find(known & ~done);
    if isempty(left)&&isempty(pending), break; end

```

```

% 与问题三/基础款一致：待清除源和剩余搜索站统一做开放路径排序。
tasks=[centers(:,pending),stations(:,left)];
tour=zeros(1,size(tasks,2)); pool=1:size(tasks,2); q=pos;
for routeStep=1:numel(tour)
    [~,pick]=min(vecnorm(tasks(:,pool)-q));
    tour(routeStep)=pool(pick); q=tasks(:,pool(pick)); pool(pick)=[];
end
improved=true;
while improved
    improved=false; route=[pos,tasks(:,tour)];
    for swapStart=1:numel(tour)-1
        for swapEnd=swapStart+1:numel(tour)
            old=norm(route(:,swapStart)-route(:,swapStart+1));
            new=norm(route(:,swapStart)-route(:,swapEnd+1));
            if swapEnd<numel(tour)
                old=old+norm(route(:,swapEnd+1)-route(:,swapEnd+2));
                new=new+norm(route(:,swapStart+1)-route(:,swapEnd+2));
            end
            if new<old-1e-7
                tour(swapStart:swapEnd)=fliplr(tour(swapStart:swapEnd));
                improved=true; route=[pos,tasks(:,tour)];
            end
        end
    end
    next=tour(1);
    if next<=numel(pending)
        k=pending(next); inLocal=true; localize(k); inLocal=false; continue;
    end
    stationId=left(next-numel(pending)); q=stations(:,stationId); visited(stationId)=true;

% 全局扫描条件与基础款相同，保留所有顺路获得方位线的机会。
list=find(~done & (~known | obsCount<6));
order=[channel,setdiff(list,channel,'stable')];
if ~ismember(channel,list), order=list; end
for k=order
    observe(q,k,false);
    if sum(done)==16, break; end
end
end
if toc(clock0)<=limit-5, act('/exit'); end
out=struct('cleared',sum(done),'totalTime',vt,'averageTime',vt/max(1,sum(done)), ...
    'complete',(all(visited)||sum(done)==16) && all(~known|done), ...
    'reason',reason,'runtime',toc(clock0),'discoveryTime',discovered, ...
    'clearTime',cleared,'observations',obsCount,'finalRadius',radii,'history',{history}, ...
    'stationsVisited',sum(visited),'stationCount',size(stations,2), ...
    'fallbackCount',fallbackCount,'raySweepCount',raySweepCount, ...

```

```

'globalDistance',globalDistance,'localDistance',localDistance, ...
'directionalModeCount',sum(mode==2),'certifiedDirectionalCount',sum(certifiedDirectional),
...
'p2Tests',omniRefineCount);
if ~local
stamp=char(datetime('now','Format','yyyyMMdd_HH:mm:ss'));
save(fullfile(root,['problem4_split_run_' stamp '.mat']),'out');
fprintf('问题四分流版: 清除%d个, 总时间%.2f s, 平均%.2f s, 完整排查=%d\n', ...
        out.cleared,out.totalTime,out.averageTime,out.complete);
end

function localize(k)
    if done(k), return; end
    if radii(k)<=20 && clearAt(centers(:,k),k), return; end

    % 分流改动: 尚未出现过失联时, 先假设为全向源, 直接复用问题三
    % “区域中心+小幅侧移”的定位形式; 点位落在1000 m保证区时,
    % 一旦无信号便可严格确认定向, 并立即退出本段。
    if mode(k)==1 && radii(k)<=150
        for omniStep=1:2
            if done(k)||mode(k)==2, break; end
            c=centers(:,k); r=radii(k);
            if r<=60
                if clearAt(c,k), return; end
                hit=observe(pos,k,true);
            else
                u=c-pos; if norm(u)<1e-9, u=[1;0]; end
                u=u/norm(u); side=min(80,r/2);
                q=c+side*[-u(2);u(1)];
                hit=observe(q,k,r+side<=1000);
            end
            omniRefineCount=omniRefineCount+1;
            if ~hit, break; end
            if radii(k)<=20 && clearAt(centers(:,k),k), return; end
        end
    end
    if done(k), return; end

    % 定向稳健段沿用基础款形式: 在测向线两侧成对前探。
    for round=1:5
        if done(k), return; end
        if radii(k)<=20 && clearAt(centers(:,k),k), return; end
        q0=lastPos(:,k); th=lastTheta(k);
        if any(~isfinite(q0))||~isfinite(th), break; end
        U=min(1500,max(vecnorm(regions{k}-q0)));
        if U<=600, raySweep(k,q0,th,U); return; end
        u=[cos(th);sin(th)]; v=[-u(2);u(1)];
    end
end

```

```

advance=min(490,0.6*U); side=advance*tan(deg2rad(1.005))+3;
probes=[q0+advance*u+side*v,q0+advance*u-side*v];
if norm(probes(:,2)-pos)<norm(probes(:,1)-pos), probes=fliplr(probes); end
hit=observe(probes(:,1),k,false);
if ~done(k)&&~hit, hit=observe(probes(:,2),k,false); end
if done(k), return; end
if ~hit
    mode(k)=2;
    raySweep(k,q0,th,advance/cos(deg2rad(1.005))+1); return;
end
if clearAt(lastPos(:,k),k), return; end
end
if done(k), return; end

fallbackCount=fallbackCount+1;
u=[cos(firstTheta(k));sin(firstTheta(k))]; v=[-u(2);u(1)];
xs=0:28:1512; ys=[-42,-14,14,42];
paths=cell(1,4); d0=inf(1,4); c=0;
for flipY=0:1
    yy=ys; if flipY, yy=fliplr(yy); end
    for flipX=0:1
        c=c+1; P=zeros(2,numel(xs)*numel(yy)); t=0;
        for r=1:numel(yy)
            xx=xs; if mod(r+flipX,2)==0, xx=fliplr(xx); end
            Q=firstPos(:,k)+u*xx+v*(yy(r)*ones(size(xx)));
            P(:,t+(1:numel(xx)))=Q; t=t+numel(xx);
        end
        paths{c}=P; d0(c)=norm(P(:,1)-pos);
    end
end
[~,c]=min(d0); P=paths{c};
for fallbackPoint=1:size(P,2)
    if clearAt(P(:,fallbackPoint),k), return; end
end
error('光学窄带兜底失败: 请检查测向误差或模拟器规则。');
end

function raySweep(k,q0,th,U)
    raySweepCount=raySweepCount+1; epsA=deg2rad(1.005);
    halfWidth=U*sin(epsA); assert(halfWidth<20, '测向带过宽, 不能单线覆盖。');
    step=1.98*sqrt(20^2-halfWidth^2);
    xs=step/2:step:U+step/2; u=[cos(th);sin(th)]; P=q0+u*xs;
    if norm(P(:,end)-pos)<norm(P(:,1)-pos), P=fliplr(P); end
    for sweepPoint=1:size(P,2)
        if clearAt(P(:,sweepPoint),k), return; end
    end
    error('测向带直线覆盖失败: 请检查误差上界或距离上界。');
end

```

```

end

function hit=observe(q,k,rangeCertified)
    if known(k) && ~rangeCertified, rangeCertified=rangeGuaranteed(q,k); end
    z=act('/measure',q,k); hit=~strcmp(z.measure_result,'no_signal');
    if ~hit
        if known(k) && mode(k)==1
            % 用户提出的“先全向、出现时有时无再按定向处理”。普通
            % 失联只改变求解分支；保证在接收距离内时还可严格确认定向。
            mode(k)=2;
            if rangeCertified, certifiedDirectional(k)=true; end
        end
        return;
    end
    if ~known(k)
        known(k)=true; mode(k)=1; discovered(k)=vt; regions{k}=domain;
    end
    if strcmp(z.measure_result,'near')
        assert(clearAt(q,k),'near 后光学清除失败。'); return;
    end
    th=deg2rad(z.svd_deg); obsCount(k)=obsCount(k)+1;
    lastPos(:,k)=q; lastTheta(k)=th;
    if obsCount(k)==1, firstPos(:,k)=q; firstTheta(k)=th; end
    [n1,b1,n2,b2]=wedge_planes(q,th,deg2rad(1.005));
    V=clip_halfplane(regions{k},n1,b1+1e-7);
    V=clip_halfplane(V,n2,b2+1e-7);
    u=[cos(th);sin(th)]; V=clip_halfplane(V,u,u'*q+1500+1e-7);
    assert(~isempty(V),'交会区域为空：观测与误差模型不相容。');
    regions{k}=V; D=diameter_many({V});
    if D<=20, c=V(:,1); r=D;
    else, [c,r]=welzl_mec(V,cfg); end
    centers(:,k)=c; radii(k)=max(r,max(vecnorm(V-c)));
end

function yes=rangeGuaranteed(q,k)
    % 问题二思想的在线判据：若Q距整个可能区域均不超过1000 m,
    % 或Q对所有候选位置都不比上次有信号点远，则全向源必有信号。
    V=regions{k}; yes=max(vecnorm(V-q))<=1000+1e-7;
    if ~yes && all(isfinite(lastPos(:,k)))
        s=lastPos(:,k);
        yes=max(2*(s-q)'*V+q'*q-s'*s)<=1e-7;
    end
end

function success=clearAt(q,k)
    z=act('/clear',q,k); success=strcmp(z.clear_result,'success');
    if success, done(k)=true; cleared(k)=vt; end
end

```

```

end

function z=act(path,q,k)
    if ~strcmp(path,'/exit')&&toc(clock0)>limit-5
        error('现实时间不足，停止发送新动作。');
    end
    serial=serial+1;
    request=struct('arena_id','default','robot_id',robotId, ...
        'request_id',sprintf('p4split-%d',serial));
    if nargin>1
        request.position=struct('x',q(1),'y',q(2)); request.channel=k;
    end
    for attempt=1:3
        try
            if local, z=transport(path,request);
            else, z=webwrite([baseUrl path],request,opt); end
            break;
        catch err
            if attempt==3, rethrow(err); end
        end
    end
    assert(z.accepted,'模拟器拒绝请求: %s',jsonencode(z)); vt=z.virtual_time_s;
    if nargin>1
        if inLocal, localDistance=localDistance+norm(q-pos);
        else, globalDistance=globalDistance+norm(q-pos); end
        pos=q;
    end
    if strcmp(path,'/measure'), channel=k; end
    history(end+1,:)={path,request,z};
end
end

```